

User Manual

for S32K1_S32M24X UART Driver

Document Number: UM2UARTASRR21-11 Rev0000R3.0.0 Rev. 1.0

1 Revision History	2
2 Introduction	3
2.1 Supported Derivatives	3
2.2 Overview	4
2.3 About This Manual	4
2.4 Acronyms and Definitions	5
2.5 Reference List	5
3 Driver	7
3.1 Requirements	7
3.2 Driver Design Summary	7
3.3 Hardware Resources	8
3.3.1 Physical Uart Channels:	8
3.3.2 Uart pins:	9
3.4 Deviations from Requirements	11
3.5 Driver Limitations	11
3.5.1 DMA limitation	11
3.5.2 Feature not supporting	11
3.6 Driver usage and configuration tips	12
3.6.1 How to use User notifications	12
3.6.2 How to configure Flexio Uart module	13
3.6.3 How to choose the appropriate baudrate	16
3.6.4 How to choose the appropriate timeout in general	16
3.6.5 How to enable Internal Loopback for self test.(Available only on LPUART channels)	17
3.6.6 How to set customized baudrate	17
3.6.7 How to configure DMA	19
3.6.8 How to configure Idle Interrupt	26
3.7 Runtime errors	26
3.8 Symbolic Names Disclaimer	27
3.9 Usage in non AUTOSAR context	27
4 Configuration Parameters	29
4.1 Uart	30
4.1.1 Module Uart	31
4.1.2 Container GeneralConfiguration	31
4.1.3 Parameter UartDevErrorDetect	32
4.1.4 Parameter DisableUartRuntimeErrorDetect	32
4.1.5 Parameter UartMultipartitionSupport	33
4.1.6 Parameter UartEnableUserModeSupport	33
4.1.7 Parameter UartTimeoutMethod	34
4.1.8 Parameter UartTimeoutDuration	34
4.1.9 Parameter UartDmaEnable	35
4.1.10 Parameter UartVersionInfoApi	35
4.1.11 Parameter UartCallbackCapability	35
4.1.12 Parameter UartCallback	36
4.1.13 Reference UartEcucPartitionRef	36
4.1.14 Container UartGlobalConfig	37
4.1.15 Container UartChannel	37
4.1.16 Parameter UartHwUsing	37
4.1.17 Parameter UartChannelId	38
4.1.18 Reference UartClockRef	38
4.1.19 Reference UartChannelEcucPartitionRef	39
4.1.20 Container DetailModuleConfiguration	39

4.1.21	Parameter	UartHwChannel	39
4.1.22	Parameter	DesireBaudrate	40
4.1.23	Parameter	CustomBaudrateMantissa	40
4.1.24	Parameter	CustomBaudrateDivisor	41
4.1.25	Parameter	CustomBaudrateValue	41
4.1.26	Parameter	UartInterruptDmaMethod	42
4.1.27	Parameter	UartParityType	42
4.1.28	Parameter	UartStopBitNumber	43
4.1.29	Parameter	UartWordLength	43
4.1.30	Parameter	UartInternalLoopbackEnable	44
4.1.31	Parameter	UartTimeoutEnable	44
4.1.32	Reference	UartDmaTxChannelRef	44
4.1.33	Reference	UartDmaRxChannelRef	45
4.1.34	Container	FlexioModuleConfiguration	45
4.1.35	Parameter	FlexioUartInterruptDmaMethod	46
4.1.36	Parameter	DesireBaudrate	46
4.1.37	Parameter	CustomTimerDecrement	47
4.1.38	Parameter	CustomBaudrateDivider	47
4.1.39	Parameter	CustomBaudrateValue	48
4.1.40	Parameter	bitCount	48
4.1.41	Parameter	driverDirection	49
4.1.42	Reference	UartHwChannelRef	49
4.1.43	Reference	FlexioDmaChannelRef	49
4.1.44	Container	CommonPublishedInformation	50
4.1.45	Parameter	ArReleaseMajorVersion	50
4.1.46	Parameter	ArReleaseMinorVersion	51
4.1.47	Parameter	ArReleaseRevisionVersion	51
4.1.48	Parameter	ModuleId	51
4.1.49	Parameter	SwMajorVersion	52
4.1.50	Parameter	SwMinorVersion	52
4.1.51	Parameter	SwPatchVersion	53
4.1.52	Parameter	VendorApiInfix	53
4.1.53	Parameter	VendorId	54
4.2	LPUART_UART		54
4.2.1	Module	Lpuart_Uart	55
4.3	FLEXIO_UART		55
4.3.1	Module	Flexio_Uart	55
5	Module Index		57
5.1	Software Specification		57
6	Module Documentation		58
6.1	UART Driver		58
6.1.1	Detailed Description		58
6.1.2	Data Structure Documentation		59
6.1.2.1	struct	Uart_ChannelConfigType	59
6.1.2.2	struct	Uart_ConfigType	59
6.1.3	Enum Reference		60
6.1.3.1	Uart_DrvStatusType		60
6.1.3.2	Uart_DataDirectionType		60
6.1.3.3	Uart_StatusType		60
6.1.4	Function Reference		61
6.1.4.1	Uart_Init()		61
6.1.4.2	Uart_Deinit()		61

6.1.4.3	Uart_SetBaudrate()	62
6.1.4.4	Uart_GetBaudrate()	62
6.1.4.5	Uart_SetBuffer()	63
6.1.4.6	Uart_SyncSend()	63
6.1.4.7	Uart_SyncReceive()	64
6.1.4.8	Uart_Abort()	65
6.1.4.9	Uart_AsyncSend()	65
6.1.4.10	Uart_AsyncReceive()	66
6.1.4.11	Uart_GetStatus()	66
6.2	Flexio UART IPL	68
6.2.1	Detailed Description	68
6.2.2	Data Structure Documentation	68
6.2.2.1	struct Flexio_Uart_Ip_UserConfigType	68
6.2.3	Macro Definition Documentation	69
6.2.3.1	FLEXIO_HW_INSTANCE	69
6.2.4	Enum Reference	69
6.2.4.1	Flexio_Uart_IP_DriverType	69
6.2.4.2	Flexio_Uart_Ip_DirectionType	69
6.2.4.3	Flexio_Uart_Ip_EventType	70
6.2.4.4	Flexio_Uart_Ip_StatusType	70
6.2.4.5	Flexio_Uart_Ip_BaudrateType	71
6.2.4.6	Flexio_Uart_Ip_TimerDecrementType	71
6.2.4.7	Flexio_Uart_Ip_BitCountPerCharType	71
6.2.5	Variable Documentation	71
6.2.5.1	Flexio_Uart_Ip_StateStructureType	71
6.3	Lpuart UART IPL	72
6.3.1	Detailed Description	72
6.3.2	Data Structure Documentation	73
6.3.2.1	struct Lpuart_Uart_Ip_StateStructureType	73
6.3.2.2	struct Lpuart_Uart_Ip_UserConfigType	74
6.3.3	Types Reference	74
6.3.3.1	Lpuart_Uart_Ip_CallbackType	74
6.3.4	Enum Reference	74
6.3.4.1	Lpuart_Uart_Ip_TransferType	74
6.3.4.2	Lpuart_Uart_Ip_StatusType	75
6.3.4.3	Lpuart_Uart_Ip_EventType	75
6.3.4.4	Lpuart_Uart_Ip_BaudrateType	76
6.3.5	Function Reference	76
6.3.5.1	Lpuart_Uart_Ip_Init()	76
6.3.5.2	Lpuart_Uart_Ip_Deinit()	76
6.3.5.3	Lpuart_Uart_Ip_SyncSend()	76
6.3.5.4	Lpuart_Uart_Ip_AsyncSend()	77
6.3.5.5	Lpuart_Uart_Ip_GetTransmitStatus()	77
6.3.5.6	Lpuart_Uart_Ip_AbortSendingData()	78
6.3.5.7	Lpuart_Uart_Ip_SyncReceive()	78
6.3.5.8	Lpuart_Uart_Ip_AsyncReceive()	79
6.3.5.9	Lpuart_Uart_Ip_GetReceiveStatus()	79
6.3.5.10	Lpuart_Uart_Ip_AbortReceivingData()	80
6.3.5.11	Lpuart_Uart_Ip_SetBaudRate()	80
6.3.5.12	Lpuart_Uart_Ip_GetBaudRate()	81
6.3.5.13	Lpuart_Uart_Ip_SetTxBuffer()	81
6.3.5.14	Lpuart_Uart_Ip_SetRxBuffer()	81



Chapter 1

Revision History

Revision	Date	Author	Description
1.0	07.03.2025	NXP RTD Team	S32K1_S32M24X Real-Time Drivers AUTOSAR R21-11 Version 3.0.0

Chapter 2

Introduction

- [Supported Derivatives](#)
- [Overview](#)
- [About This Manual](#)
- [Acronyms and Definitions](#)
- [Reference List](#)

This User Manual describes NXP Semiconductor CDD UART for S32K1_S32M24X. CDD UART driver configuration parameters and deviations from the specification are described in Driver chapter of this document.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32k116_qfn32
- s32k116_lqfp48
- s32k118_lqfp48
- s32k118_lqfp64
- s32k142_lqfp48
- s32k142_lqfp64
- s32k142_lqfp100
- s32k142w_lqfp48
- s32k142w_lqfp64
- s32k144_lqfp48
- s32k144_lqfp64 / MWCT1014S_lqfp64
- s32k144_lqfp100 / MWCT1014S_lqfp100
- s32k144_mapbga100
- s32k144w_lqfp48
- s32k144w_lqfp64
- s32k146_lqfp64

- s32k146_lqfp100 / MWCT1015S_lqfp100
- s32k146_mapbga100 / MWCT1015S_mapbga100
- s32k146_lqfp144
- s32k148_lqfp100
- s32k148_mapbga100 / MWCT1016S_mapbga100
- s32k148_lqfp144
- s32k148_lqfp176
- s32m241_lqfp64
- s32m242_lqfp64
- s32m243_lqfp64
- s32m244_lqfp64

All of the above microcontroller devices are collectively named as S32K1_S32M24X. Note: MWCT part numbers contain NXP confidential IP for Qi Wireless Power

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSMI	Basic Software Make file Interface
CS	Chip Select
DEM	Diagnostic Event Manager
DET	Development Error Tracer
DMA	Direct Memory Access
ECU	Electronic Control Unit
FIFO	First In First Out
LSB	Least Significant Bit
MCU	Micro Controller Unit
MIDE	Multi Integrated Development Environment
MSB	Most Significant Bit
N/A	Not Applicable
RAM	Random Access Memory
SIU	Systems Integration Unit
SWS	Software Specification
UART	Universal asynchronous receiver-transmitter
XML	Extensible Markup Language
BSW	Basic Software
ISR	Interrupt Service Routine
OS	Operating System
GUI	Graphical User Interface
PB Variant	Post Build Variant
PC Variant	Pre Compile Variant
LT Variant	Link Time Variant

2.5 Reference List

#	Title	Version
1	S32K1xx Reference Manual	Rev.14.1, 01/2024
2	S32M24x Reference Manual	Rev. 5, 12/2024
3	S32K1xx Data Sheet	Rev. 14, 08/2021
4	S32M2xx Data Sheet	Rev. 7, 12/2024
5	S32K116 Errata	Mask Set Errata for Mask 0N96V Rev. 17/JAN/2024, 1/2024
6	S32K118 Errata	Mask Set Errata for Mask 0N97V Rev. 15/JAN/2024, 1/2024
7	S32K142 Errata	Mask Set Errata for Mask 0N33V Rev. 15/JAN/2024, 1/2024
8	S32K144 Errata	Mask Set Errata for Mask 0N57U Rev. 15/JAN/2024, 1/2024
9	S32K144W Errata	Mask Set Errata for Mask 0P64A Rev. 03/JUL/2024, 7/2024
10	S32K146 Errata	Mask Set Errata for Mask 0N73V Rev. 15/JAN/2024, 1/2024
11	S32K148 Errata	Mask Set Errata for Mask 0N20V Rev. 15/JAN/2024, 1/2024

Introduction

#	Title	Version
12	S32M242 Errata	Mask Set Errata for Mask N33V+P69K Rev.1, 8/2024
13	S32M244 Errata	Mask Set Errata for Mask P64A+P69K Rev.1, 8/2024

Chapter 3

Driver

- [Requirements](#)
- [Driver Design Summary](#)
- [Hardware Resources](#)
- [Deviations from Requirements](#)
- [Driver Limitations](#)
- [Driver usage and configuration tips](#)
- [Runtime errors](#)
- [Symbolic Names Disclaimer](#)
- [Usage in non AUTOSAR context](#)

3.1 Requirements

UART is a Complex Device Driver (CDD), so there are no AUTOSAR requirements regarding this module.

It has vendor-specific requirements and implementation.

3.2 Driver Design Summary

The Uart driver is part of the Real Time Drivers, performs the hardware access and offers a hardware independent API to the application. The Uart driver is implemented as an Autosar complex device driver. It uses the Uart hardware peripheral which provides support for implementing the Uart protocol. Hardware and software settings can be configured using an Autosar standard configuration tool. The driver reports errors to the error manager as defined in AUTOSAR. The following features are implemented in the Uart Module for S32K1_S32M24X device:

- Full-duplex communication
- Configurable baud-rate. It supports the standard UART baudrates. The values can be checked in the Uart↔_BaudrateType enum.
- Configurable parity, stop bits
- DMA transfers
- Callback notifications
- Continuous transfers in an asynchronous manner

3.3 Hardware Resources

3.3.1 Physical Uart Channels:

The hardware instances configured by the Uart driver:

- From Lpuart_0 to Lpuart_2 with S32K144, S32K142W, S32K144W, S32K146, S32K148.
- From Lpuart_0 to Lpuart_1 with S32K116, S32K118, S32K142.
- From Lpuart_0 to Lpuart_1 with S32M241, S32M242, S32M243, S32M244.

The Flexio Channel configured by the Uart driver:

- From Flexio_0 to Flexio_3 with S32K116, S32K118, S32K142, S32K144, S32K142W, S32K144W, S32K146, S32K148.
- From Flexio_0 to Flexio_3 with S32M241, S32M242, S32M243, S32M244.

Note: In EB tresos, Lpuart Uart Physical Unit has selected by UartHwChannel ("General" tab).

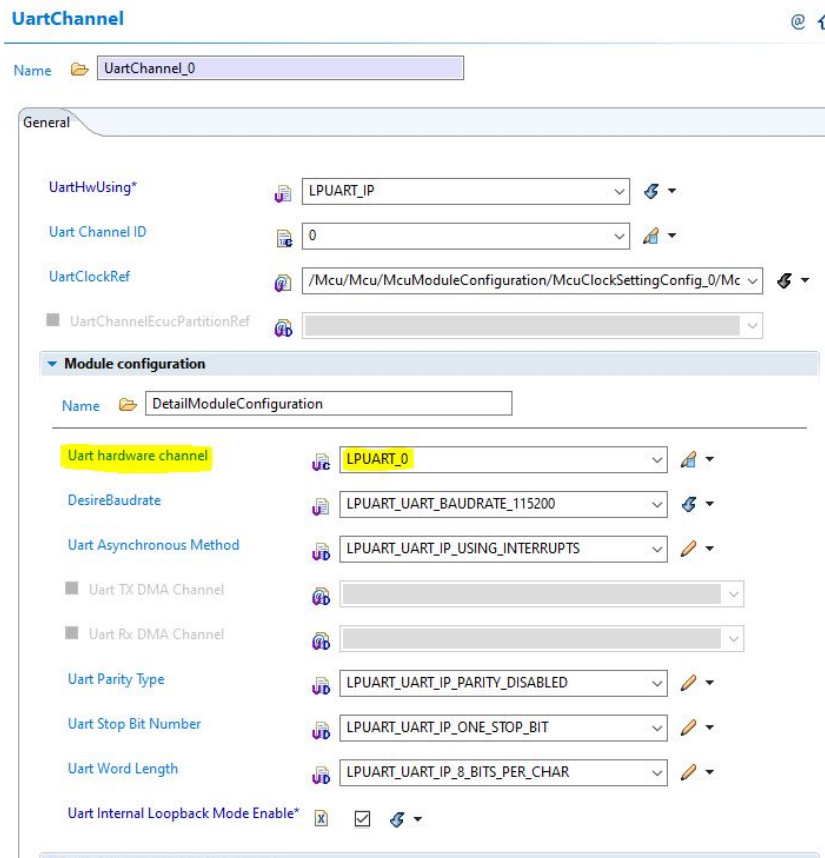


Figure 3.1 Uart Physical Unit has selected by UartHwChannel in EB Tresos

Flexio Uart Physical Channel has selected by MCL module at Mcl/MclConfig/FlexioCommon/FlexioCommon_0/FlexioMclLogicChannels path

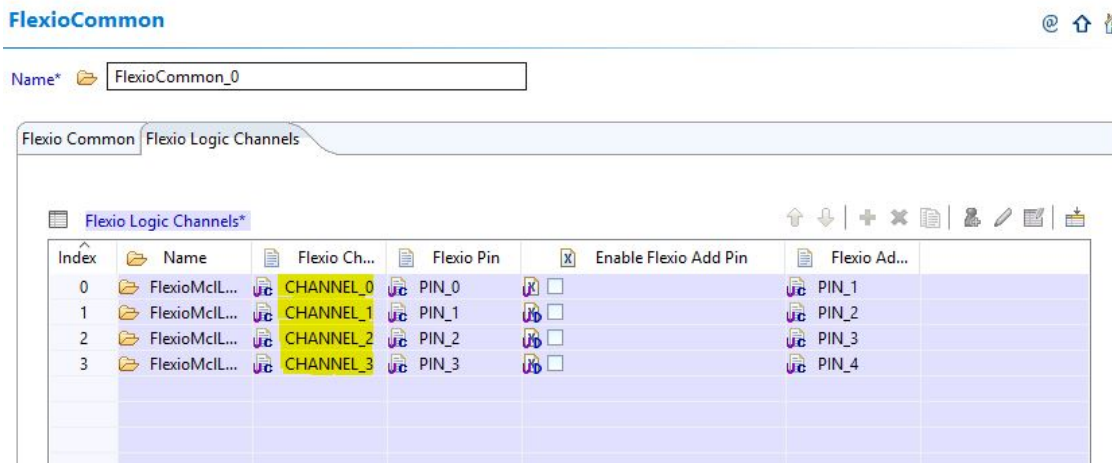


Figure 3.2 Flexio Uart Physical Channel has selected by MCL module in EB Tresos

3.3.2 Uart pins:

- The pins of S32K1xx can be found out in the file "S32K1xx_IO_Signal_Description_Input_Multiplexing.xlsx" from attached file of S32K1XX Reference Manual Reference List. The Lpuart_0, Lpuart_1 and Lpuart_2 use the pins LPUART0, LPUART1 and LPUART2 correspondingly.
- The pins of S32M24x can be found out in the file "S32M24x_IOMUX.xlsx" from attached file of S32M24x Reference Manual Reference List. The Lpuart_0 and Lpuart_1 use the pins LPUART0 and LPUART1 correspondingly.
- Example for S32K148 the pins can be found in "S32K148_IO_Signal_Description_Input_Multiplexing.xlsx" as blow:
 - For Lpuart channel:

Port	CR	SSS	Function	Module	Description	Direction
PTA0	PCR_PTA0	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0110	LPUART0_CTS	LPUART0	Clear To Send (bar)	I
PTA1	PCR_PTA1	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0110	LPUART0_RTS	LPUART0	Request To Send	O
PTA2	PCR_PTA2	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0110	LPUART0_RX	LPUART0	Receive	I
PTA3	PCR_PTA3	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0110	LPUART0_TX	LPUART0	Transmit	I/O
PTA27	PCR_PTA27	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0100	LPUART0_TX	LPUART0	Transmit	I/O
PTA28	PCR_PTA28	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0100	LPUART0_RX	LPUART0	Receive	I
PTB0	PCR_PTBO	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0010	LPUART0_RX	LPUART0	Receive	I
PTB1	PCR_PTBO	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0010	LPUART0_TX	LPUART0	Transmit	I/O
PTC2	PCR_PTC2	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0100	LPUART0_RX	LPUART0	Receive	I
PTC3	PCR_PTC3	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0100	LPUART0_TX	LPUART0	Transmit	I/O
PTC8	PCR_PTC8	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0110	LPUART0_CTS	LPUART0	Clear To Send (bar)	I
PTC9	PCR_PTC9	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0110	LPUART0_RTS	LPUART0	Request To Send	O

Figure 3.3 Pins selected for LPUART0 of S32K148

Port	CR	SSS	Function	Module	Description	Direction
PTA6	PCR_PTA6	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0110	LPUART1_CTS	LPUART1	Clear To Send (bar)	I
PTA7	PCR_PTA7	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0110	LPUART1_RTS	LPUART1	Request To Send	O
PTA18	PCR_PTA18	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0011	LPUART1_TX	LPUART1	Transmit	I/O
PTA19	PCR_PTA19	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0011	LPUART1_RX	LPUART1	Receive	I
PTB22	PCR_PTB22	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0101	LPUART1_TX	LPUART1	Transmit	I/O
PTB23	PCR_PTB23	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0011	LPUART1_RX	LPUART1	Receive	I
PTC6	PCR_PTC6	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0010	LPUART1_RX	LPUART1	Receive	I
PTC7	PCR_PTC7	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0010	LPUART1_TX	LPUART1	Transmit	I/O
PTC8	PCR_PTC8	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0010	LPUART1_RX	LPUART1	Receive	I
PTC9	PCR_PTC9	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0010	LPUART1_TX	LPUART1	Transmit	I/O
PTD13	PCR_PTD13	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0011	LPUART1_RX	LPUART1	Receive	I
PTD14	PCR_PTD14	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0011	LPUART1_TX	LPUART1	Transmit	I/O
PTE2	PCR_PTE2	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0110	LPUART1_CTS	LPUART1	Clear To Send (bar)	I
PTE6	PCR_PTE6	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0110	LPUART1_RTS	LPUART1	Request To Send	O
PTE15	PCR_PTE15	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0010	LPUART1_CTS	LPUART1	Clear To Send (bar)	I
PTE16	PCR_PTE16	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0010	LPUART1_RTS	LPUART1	Request To Send	O

Figure 3.4 Pins selected for LPUART1 of S32K148

Port	CR	SSS	Function	Module	Description	Direction
PTA8	PCR_PTA8	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0010	LPUART2_RX	LPUART2	Receive	I
PTA9	PCR_PTA9	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0010	LPUART2_TX	LPUART2	Transmit	I/O
PTA29	PCR_PTA29	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0100	LPUART2_TX	LPUART2	Transmit	I/O
PTA30	PCR_PTA30	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0011	LPUART2_RX	LPUART2	Receive	I
PTC12	PCR_PTC12	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0100	LPUART2_CTS	LPUART2	Clear To Send (bar)	I
PTC13	PCR_PTC13	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0100	LPUART2_RTS	LPUART2	Request To Send	O
PTD6	PCR_PTD6	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0010	LPUART2_RX	LPUART2	Receive	I
PTD7	PCR_PTD7	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0010	LPUART2_TX	LPUART2	Transmit	I/O
PTD11	PCR_PTD11	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0110	LPUART2_CTS	LPUART2	Clear To Send (bar)	I
PTD12	PCR_PTD12	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0110	LPUART2_RTS	LPUART2	Request To Send	O
PTD17	PCR_PTD17	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0011	LPUART2_RX	LPUART2	Receive	I
PTE3	PCR_PTE3	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0011	LPUART2_RTS	LPUART2	Request To Send	O
PTE9	PCR_PTE9	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0011	LPUART2_CTS	LPUART2	Clear To Send (bar)	I
PTE12	PCR_PTE12	0000_0000	DISABLED		Signal Path Disabled	-
		0000_0011	LPUART2_TX	LPUART2	Transmit	I/O

Figure 3.5 Pins selected for LPUART2 of S32K148

– For Flexio channel: Each Flexio channel uses one of the pin in list FXIO_D0, FXIO_D1,..., FXIO_D3.

Po	CR	SSS	Function	Module	Description	Action
PTA0		0000_0100	FXIO_D2	FXIO	FlexIO Bi-directional Shift/timer I/O	I/O
PTA1		0000_0100	FXIO_D3	FXIO	FlexIO Bi-directional Shift/timer I/O	I/O
PTA2		0000_0101	FXIO_D4	FXIO	FlexIO Bi-directional Shift/timer I/O	I/O
PTA3		0000_0101	FXIO_D5	FXIO	FlexIO Bi-directional Shift/timer I/O	I/O
PTA8		0000_0100	FXIO_D6	FXIO	FlexIO Bi-directional Shift/timer I/O	I/O
PTA9		0000_0100	FXIO_D7	FXIO	FlexIO Bi-directional Shift/timer I/O	I/O
PTA10		0000_0100	FXIO_D0	FXIO	FlexIO Bi-directional Shift/timer I/O	I/O
PTA11		0000_0100	FXIO_D1	FXIO	FlexIO Bi-directional Shift/timer I/O	I/O
PTA21		0000_0011	FXIO_D0	FXIO	FlexIO Bi-directional Shift/timer I/O	I/O
PTA22		0000_0011	FXIO_D1	FXIO	FlexIO Bi-directional Shift/timer I/O	I/O
PTA23		0000_0011	FXIO_D2	FXIO	FlexIO Bi-directional Shift/timer I/O	I/O
PTA24		0000_0011	FXIO_D3	FXIO	FlexIO Bi-directional Shift/timer I/O	I/O
PTC30		0000_0011	FXIO_D0	FXIO	FlexIO Bi-directional Shift/timer I/O	I/O
PTC31		0000_0011	FXIO_D1	FXIO	FlexIO Bi-directional Shift/timer I/O	I/O
PTD0		0000_0110	FXIO_D0	FXIO	FlexIO Bi-directional Shift/timer I/O	I/O
PTD1		0000_0110	FXIO_D1	FXIO	FlexIO Bi-directional Shift/timer I/O	I/O
PTD2		0000_0100	FXIO_D4	FXIO	FlexIO Bi-directional Shift/timer I/O	I/O
PTD2		0000_0101	FXIO_D6	FXIO	FlexIO Bi-directional Shift/timer I/O	I/O
PTD3		0000_0100	FXIO_D5	FXIO	FlexIO Bi-directional Shift/timer I/O	I/O
PTD3		0000_0101	FXIO_D7	FXIO	FlexIO Bi-directional Shift/timer I/O	I/O
PTD8		0000_0101	FXIO_D1	FXIO	FlexIO Bi-directional Shift/timer I/O	I/O
PTD9		0000_0011	FXIO_D0	FXIO	FlexIO Bi-directional Shift/timer I/O	I/O
PTD18		0000_0011	FXIO_D2	FXIO	FlexIO Bi-directional Shift/timer I/O	I/O
PTD19		0000_0011	FXIO_D3	FXIO	FlexIO Bi-directional Shift/timer I/O	I/O
PTD26		0000_0011	FXIO_D7	FXIO	FlexIO Bi-directional Shift/timer I/O	I/O
PTD31		0000_0011	FXIO_D6	FXIO	FlexIO Bi-directional Shift/timer I/O	I/O
PTE4		0000_0110	FXIO_D6	FXIO	FlexIO Bi-directional Shift/timer I/O	I/O
PTE5		0000_0110	FXIO_D7	FXIO	FlexIO Bi-directional Shift/timer I/O	I/O
PTE10		0000_0110	FXIO_D4	FXIO	FlexIO Bi-directional Shift/timer I/O	I/O
PTE11		0000_0110	FXIO_D5	FXIO	FlexIO Bi-directional Shift/timer I/O	I/O
PTE15		0000_0110	FXIO_D2	FXIO	FlexIO Bi-directional Shift/timer I/O	I/O
PTE16		0000_0110	FXIO_D3	FXIO	FlexIO Bi-directional Shift/timer I/O	I/O
PTE17		0000_0011	FXIO_D5	FXIO	FlexIO Bi-directional Shift/timer I/O	I/O
PTE18		0000_0011	FXIO_D4	FXIO	FlexIO Bi-directional Shift/timer I/O	I/O

Figure 3.6 Pins selected for Flexio of S32K148

3.4 Deviations from Requirements

The driver deviates from the AUTOSAR UART Driver software specification in some places. The table identifies the AUTOSAR requirements that are out of scope, not implemented or not fully implemented for the Uart Driver.

Term	Definition
N/S	Out of scope
N/I	Not implemented
N/F	Not fully implemented

UART is a Complex Device Driver (CDD), so there are no AUTOSAR requirements regarding this module. It has vendor-specific requirements and implementation.

3.5 Driver Limitations

3.5.1 DMA limitation

- Driver does not support DMA feature in 9-bits, 10-bits mode.

3.5.2 Feature not supporting

- FlexIO Uart does not support Loopback mode for self testing.

3.6 Driver usage and configuration tips

3.6.1 How to use User notifications

- The Uart driver can use a function callback in order to notify the application on some of the following events in a transmission: *end transfer*, *rx buffer full*, *tx buffer empty* and *error*. This notification is configured via the configurators supported on the driver: EBT and Design Studio. To enable Uart Callback, on both configurators, GeneralConfiguration/UartCallbackCapability must be set to "True".

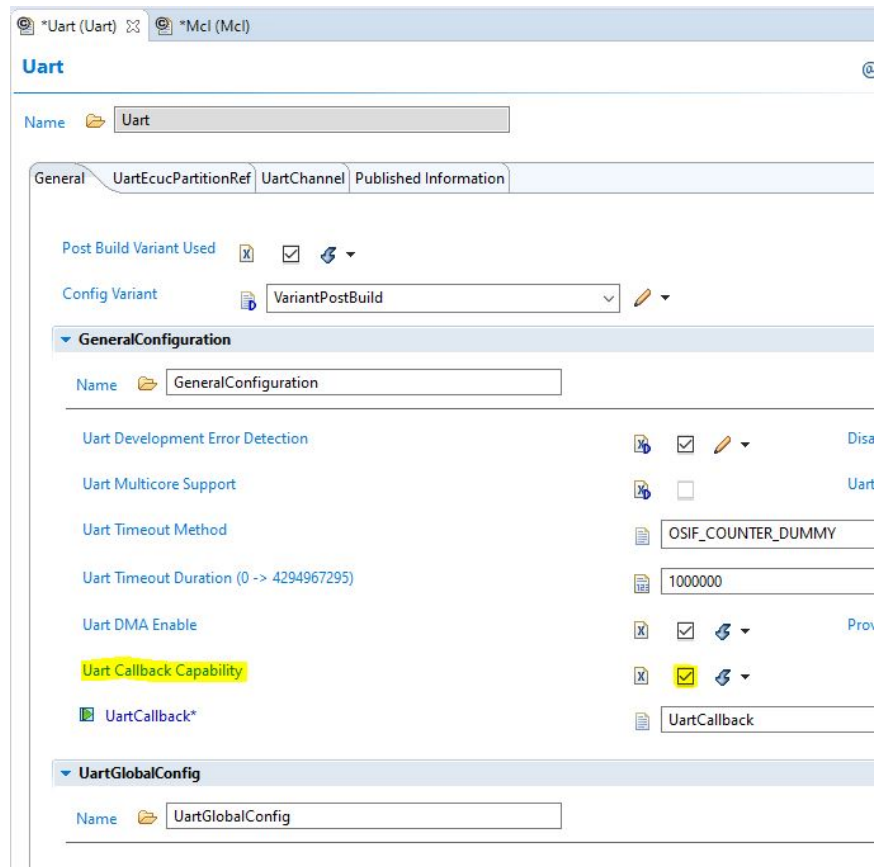


Figure 3.7 Enable Uart Callback Capability

If the HLD driver is used, then the name of the functions must be configured in the following nodes:

UartCallback*

UartCallback

Figure 3.8 User Notifications

- The signature of the functions shall follow the Uart_CallbackType type.

- If the IP driver is used, the function name can be configured in the same path, but the function signature should follow the callback type corresponding to the Hardware Instance used (Lpuart or Flexio).
- For IP driver, an extra feature can be used: a callback parameter. This variable is a parameter of the notification function and has the void* type in order to be a flexible solution for the user. This parameter can be casted to any data type and can be used to pass information from callback call to another. In order to use it in the application, define it in the application as a global variable void* callback_parameter_name. Add the variable name (callback_parameter_name) in the GeneralConfiguration container of the configurator.
- Another important callback parameter passed is the event that triggered the notification call. Its type depends on the instances of driver in use: Uart_EventType or Ip_name_Uart_Ip_EventType. Both enums have the following macros: `_EVENT_RX_FULL = 0x00U`,
`_EVENT_TX_EMPTY = 0x01U`,
`_EVENT_END_TRANSFER = 0x02U`,
`_EVENT_ERROR = 0x03U`,

Note

The first two can be used with Uart_SetBuffer or Lpuart/Flexio_Uart_Ip_Setbuffer API to replace the current buffer when it is full/empty. This approach ensures a continuous transfer.

3.6.2 How to configure Flexio Uart module

FlexIO module has three types of resources: SHIFTERS, TIMERS and PINS. These resources must be split between RTD drivers without any overlap. In order to define the solution for a correct resource allocation, we defined a channel as following: FLEXIO_CHANNEL_x - Has allocated Shifter x and Timer x.

For example: If a driver chooses FLEXIO_CHANNEL_0, Shifter 0 and Timer 0 are allocated to it. Separately, a channel has to configure a PIN.

First, MclGeneral/MclFlexioCommon/MclEnableFlexioCommon must be set to "True"

Mcl @ ↑

Name 📁 Mcl

General Mcl Virtual Memory S Dma Logic Instance Dma Logic Channel »₃

Name 📁 MclDma

Enable DMA Support 📄 ☐ ✎

▼ Mcl Cache

Name 📁 MclCache

Enable CACHE Support 📄 ☐ ✎

Cache Timeout Value (0 -> 65536) 📄 0

Cache Timeout Method 📄 OSIF_COUNTER

▼ Mcl TrgMux

Name 📁 MclTrgMux

Enable TRGMUX Support 📄 ☐ ✎

▼ Mcl Flexio Common

Name 📁 MclFlexioCommon

Enable Flexio Common Support* 📄 ☒ 🔄

Figure 3.9 Enable Flexio Common Support

Flexio Channels are configured in the Mcl component at the Mcl/MclConfig/FlexioCommon/FlexioMclLogicChannels path. Each driver shall use as many channels references from Mcl as it needs. For example Flexio Uart Transmit, requires 1 shifter, 1 timer and 1 pin, therefore this channel references will be used in the configuration.

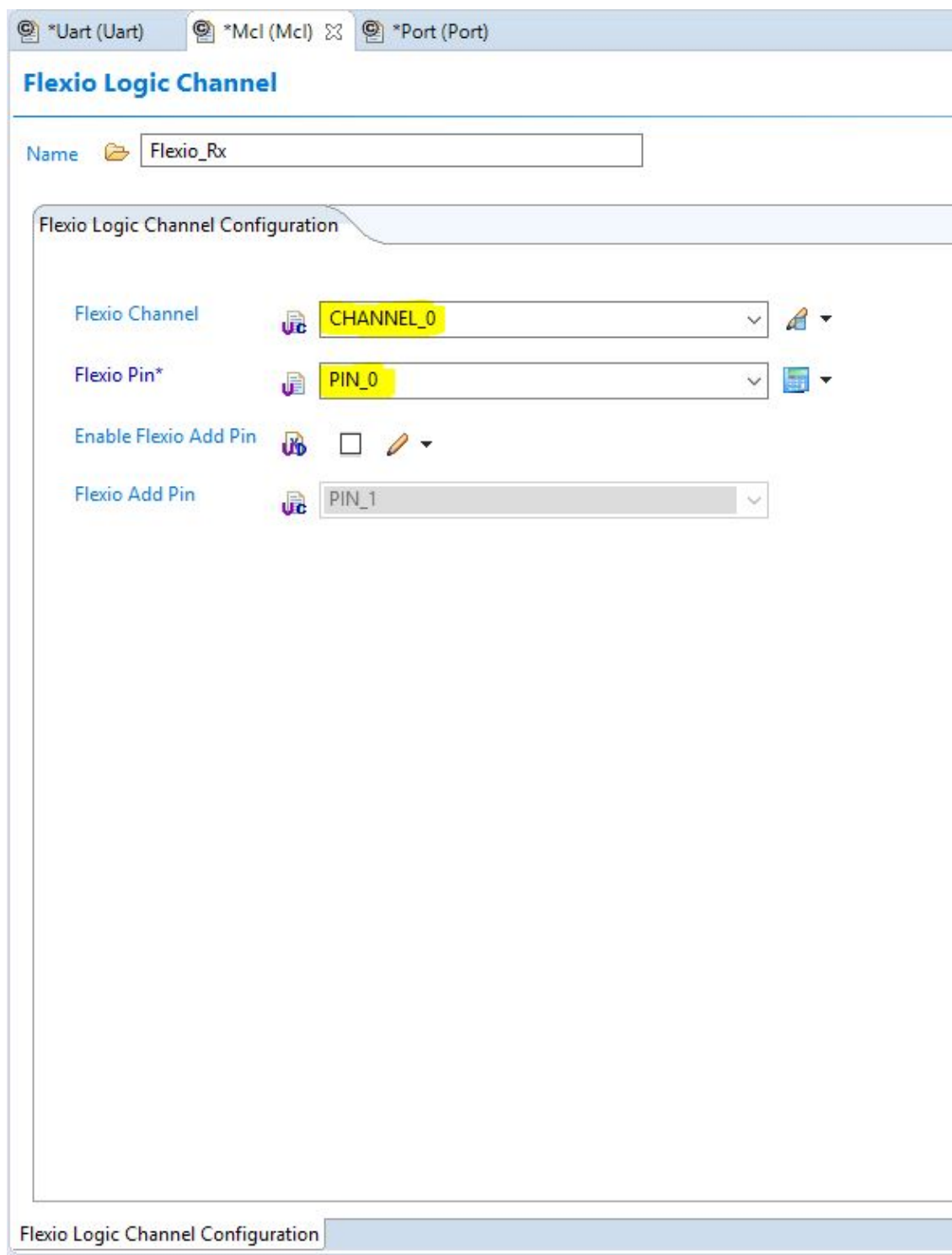


Figure 3.10 Use Flexio Channel 0 (TIMER0, SHIFTER0) and Pin_0(FXIO_D0)

Then Uart/UartGlobalConfig/UartChannel/FlexioModuleConfiguration/UartHwChannelRef must be select Flexio Channel respectively

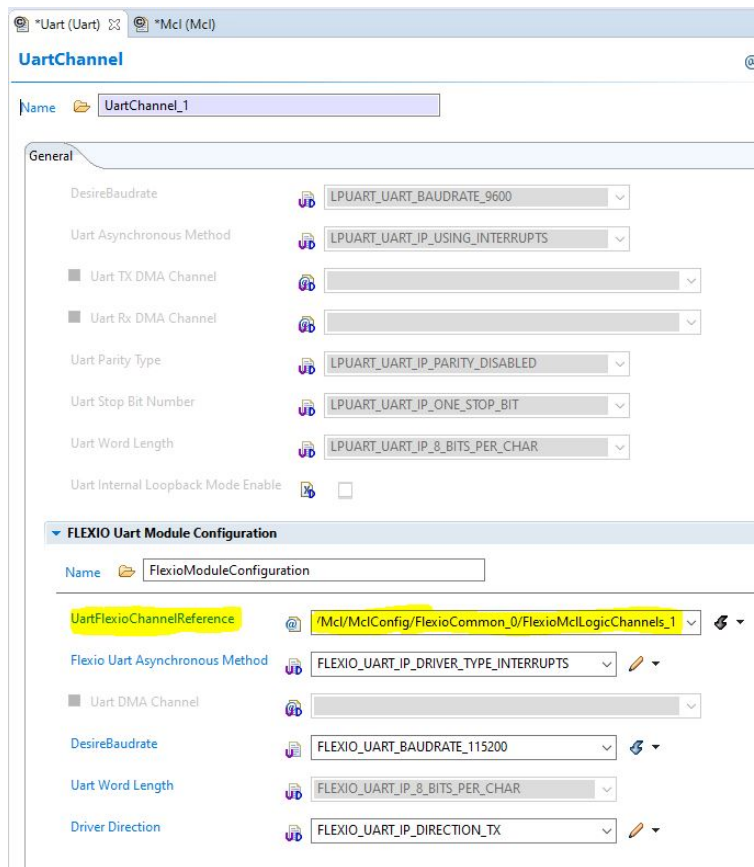


Figure 3.11 Select Flexio Channel configured in MCL module

Note: Remove any access to common registers, described in the Code guideline section. In the application/testcases, before using FlexIO, call `Mcl_Init()` function, located in `CDD_Mcl.c` source file. This API will set the module enable bit (`CTRL[FLEXEN]`). At the end of the application/testcase, call `Mcl_DeInit` function. This API will disable the FlexIO module by set to 0 `CTRL[FLEXEN]` bit. For accessing the common registers, include `Flexio_Mcl_Ip_HwAccess.h` header and use the API from it.

3.6.3 How to choose the appropriate baudrate

Check the generated user struct configuration for the actual calculated baudrate.

- The baudrate deviation should be less than 5% with the normal baudrate value (lower than 921600) for properly operation.
- For higher baudrate (921600 or 1843200), The baudrate deviation should be less than 1%.
- Flexio Baudrate divider greater than 255 it will raise an error, user need to choose a reasonable Clock or Baudrate.

3.6.4 How to choose the appropriate timeout in general

The timeout need to config in GeneralConfiguration tab of EB Tresos or S32DS tool. For normal operation, this value need to config larger than $1 \text{ Uart frame period} = (1s * (\text{number of bit per frame}) / \text{baudrate})$

3.6.5 How to enable Internal Loopback for self test.(Available only on LPUART channels)

Loop mode is sometimes used to check software, independent of connections in the external system, to help isolate system problems. In this mode, the transmitter output is internally connected to the receiver input and the RXD pin is not used by the LPUART

For enabling this mode, you can check the UartInternalLoopbackEnable for each channel that will be configured in this mode.

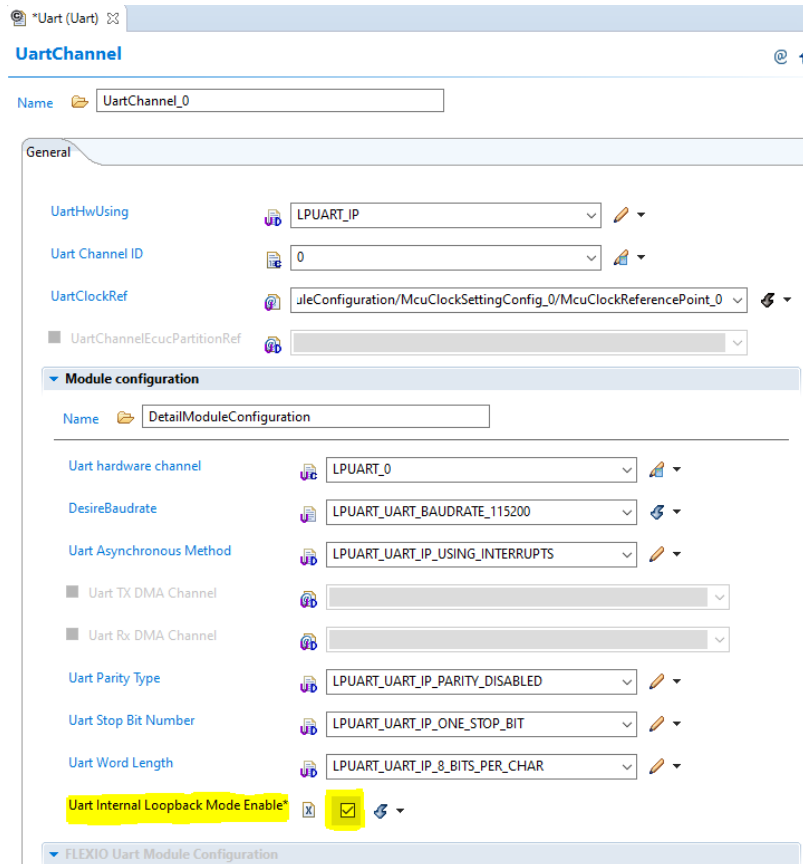


Figure 3.12 Internal Loopback mode configuration

Note: User should call Async Receive function before Async Send function to ensure that the receiver is ready to receive data before the data is transmitted.

3.6.6 How to set customized baudrate

Baudrate customization allows the user to set the any baudrate value supported by hardware for the Uart channel.

For LPUART IP:

- Select LPUART_UART_BAUDRATE_CUSTOM in DesireBaudrate field.

Driver

- Fill in the value of Custom Baudrate Mantissa (SBR) and Custom Baudrate Divisor (OSR). It will be evaluated depending on Clock Source value and user desired baudrate.

Desired baudrate = Clock Source value / (Custom Baudrate Mantissa * Custom Baudrate Divisor)

For example:

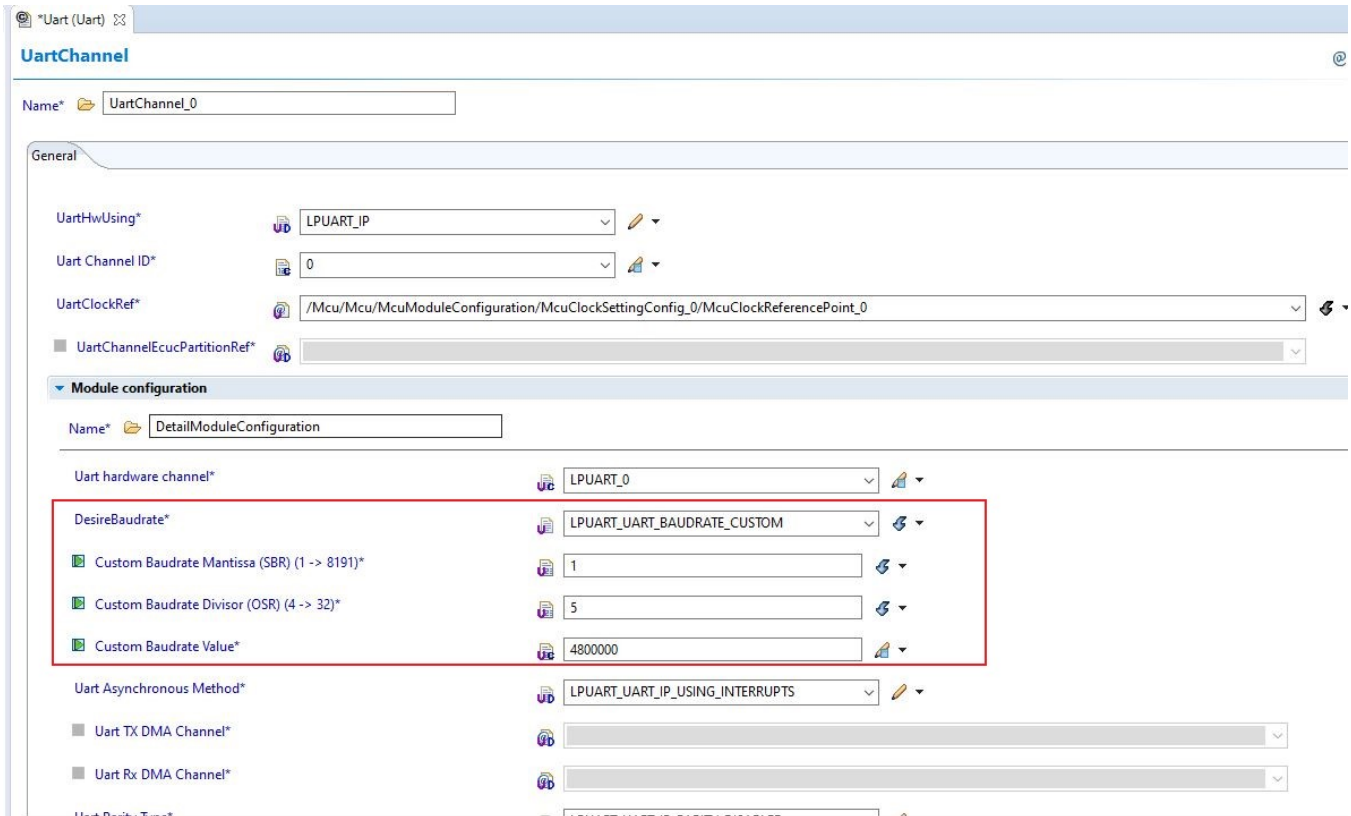


Figure 3.13 With LPUART1_CLK is 2.4E7 Hz and user desire baudrate is 4800000 Bps

Note: Press the automatic calculate value button to show the baudrate value in Custom baudrate value field.

For FLEXIO UART IP:

- Select FLEXIO_UART_BAUDRATE_CUSTOM in DesireBaurate field.
- Select the Custom Timer Decrement:
FLEXIO_TIMER_DECREMENT_FXIO_CLK_SHIFT_TMR is Decrement counter on FlexIO clock. Shift clock equals timer output.
FLEXIO_TIMER_DECREMENT_FXIO_CLK_DIV_16 is Decrement counter on FlexIO clock divided by 16. Shift clock equals timer output.(Not support on S32M24x platform)
FLEXIO_TIMER_DECREMENT_FXIO_CLK_DIV_256 is Decrement counter on FlexIO clock divided by 256. Shift clock equals timer output.(Not support on S32M24x platform)
- Fill in the value of Custom Baudrate Divider. It will be evaluated depending on Clock Source value and user desire baudrate.

Desire baudrate = Clock Source value / (Custom Timer Decrement * 2 * (Custom Baudrate Divider + 1))
 For example:

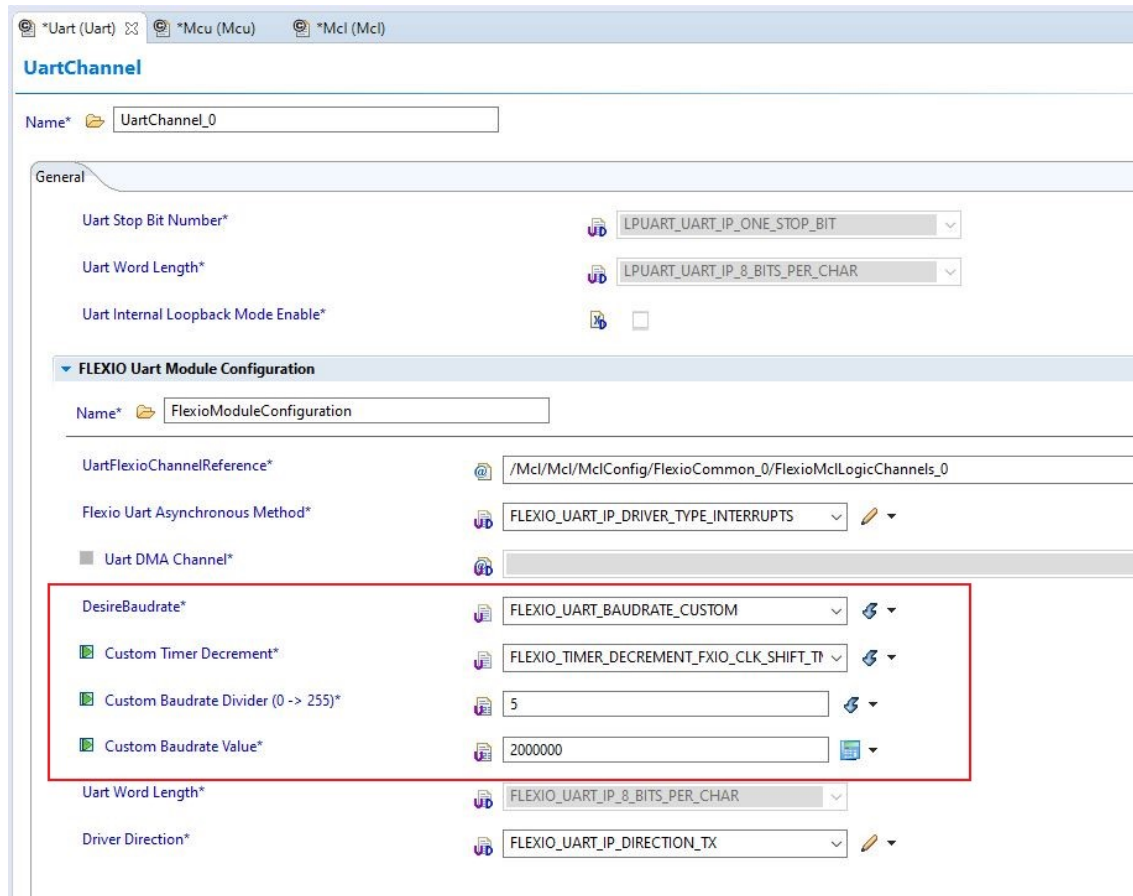


Figure 3.14 With FLEXIO_CLK is 2.4E7 Hz and user desire baudrate is 2000000 Bps

Note: Press the automatic calculate value button to show the baudrate value in Custom baudrate value field.

3.6.7 How to configure DMA

This section applies only to Uart units configured for asynchronous transmission and which use DMA for serializing/deserializing data between the hardware unit and the TX/RX buffers (UartInterruptDmaMethod = DMA).

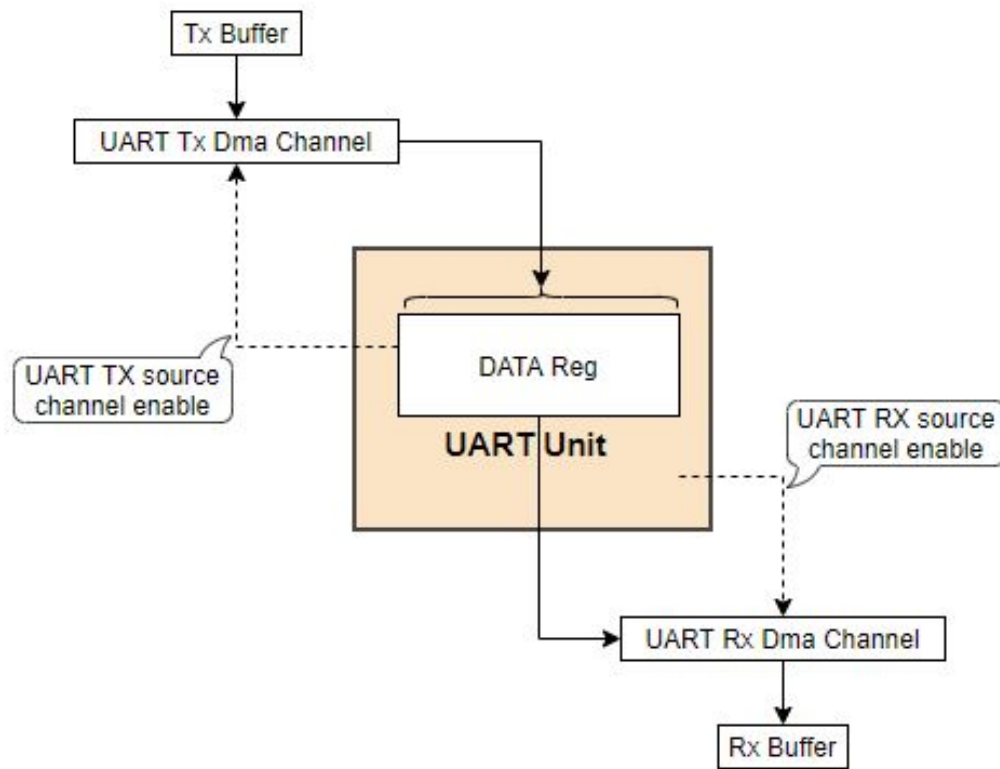


Figure 3.15 DMA transferring mode internal architecture

Each Uart unit configured in DMA mode requires 2 distinct DMA channels from the same DMA Mux:

- **Uart TX DMA Channel:** The TX DMA channel used for filling TX Data register with the data in TX Buffer. This channel is triggered by TX Uart unit event and must be wired to Uart TX source (configured inside the MCL module "McIConfig/Dma Logic Channel" container).
- **Uart RX DMA Channel:** The RX DMA channel used for filling RX buffer with the deserialized data. This channel is triggered by RX Uart unit event and must be wired to Uart RX source (configured inside the MCL module "McIConfig/Dma Logic Channel" container).

Note

- If DMA uses fixed priority arbitration, then the priority must be Uart RX DMA Channel > Uart TX DMA Channel.
- If DMA uses round robin arbitration, no priority constraints are applied on Uart TX DMA Channel, Uart RX DMA Channel priority.
- DMA asynchronous transmission is just used in 7-bits data mode and 8-bits data mode.

Next figures show an example of DMA configuration for Uart unit.

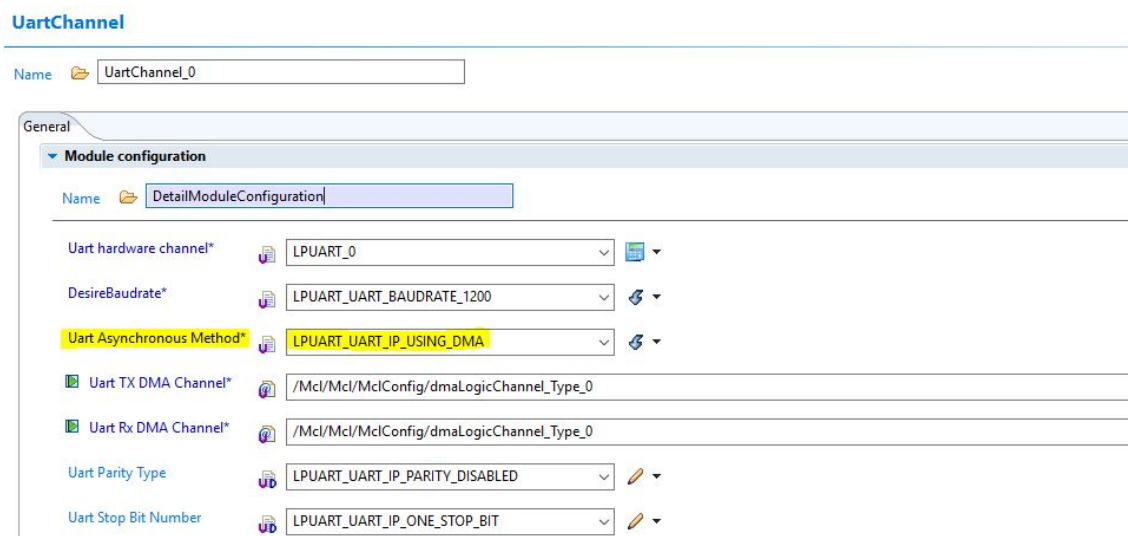


Figure 3.16 DMA Configuration sample for Uart Physical Unit - Uart module in EB Tresos

Name* Uart_Dma_Tx

Logic Channel Configuration Global Transfer ScatterGather

Logic Channel Name* DMA_LOGIC_CH_1

Hardware Instance* DMA_IP_HW_INST_0

Hardware Channel* DMA_IP_HW_CH_1

Interrupt Callback* Lpuart_0_Uart_Ip_DmaTxCompleteCallback

Error Interrupt Callback* Lpuart_0_Uart_Ip_DmaTxCompleteCallback

☐ Ecuc Partition Ref*

Enable Global Config* ☒ Enable Transfer Config* ☒

Enable Scatter/Gather* ☐

Figure 3.17 DMA TX General configuration - MCL module in EB Tresos

Logic Channel @ ↑

Name* 📁

Logic Channel Configuration **Global** Transfer ScatterGather

▼ dmaLogicChannel_GlobalConfigType

Name* 📁

▼ Request

Name* 📁

Enable DMAMUX Trigger* 📄 ☐ ✎ ▼ Enable DMAMUX Source* 📄 ☒ 🔄 ▼

DMAMUX0 Source* 📄 📄 ▼

Enable DMA Request* 📄 ☒ 🔄 ▼

▼ Interrupt

Name* 📁

Enable Error Interrupt* 📄 ☐ ✎ ▼

▼ Priority

Name* 📁

Level Priority* 📄 📄 ▼

Enable Preemption* 📄 ☐ ✎ ▼ Disable Preempt* 📄 ☐ ✎ ▼

Figure 3.18 DMA TX Global Configuration - MCL module in EB Tresos

Name*  Uart_Dma_Rx

Logic Channel Configuration Global Transfer ScatterGather

Logic Channel Name*  DMA_LOGIC_CH_0 

Hardware Instance*  DMA_IP_HW_INST_0  

Hardware Channel*  DMA_IP_HW_CH_0  

Interrupt Callback*  Lpuart_0_Uart_Ip_DmaRxCompleteCallback 

Error Interrupt Callback*  Lpuart_0_Uart_Ip_DmaRxCompleteCallback 

Ecuc Partition Ref*  

Enable Global Config*  ☒  Enable Transfer Config*  ☒ 

Enable Scatter/Gather*  ☐

Figure 3.19 DMA Rx General configuration - MCL module in EB Tresos

Logic Channel @ 

Name*  Uart_Dma_Rx

Logic Channel Configuration Global Transfer ScatterGather

dmaLogicChannel_GlobalConfigType

Name*  dmaLogicChannel_GlobalConfigType

Request

Name*  dmaLogicChannelConfig_GlobalRequestType

Enable DMAMUX Trigger*  ☐  Enable DMAMUX Source*  ☒ 

DMAMUX0 Source*  DMA_IP_REQ_MUX0_LPUART0_RX 

Enable DMA Request*  ☒ 

Interrupt

Name*  dmaLogicChannelConfig_GlobalInterruptType

Enable Error Interrupt*  ☐ 

Priority

Name*  dmaLogicChannelConfig_GlobalPriorityType

Level Priority*  DMA_IP_LEVEL_PRIO0 

Enable Preemption*  ☐  Disable Preempt*  ☐ 

Figure 3.20 DMA Rx Global Configuration - MCL module in EB Tresos

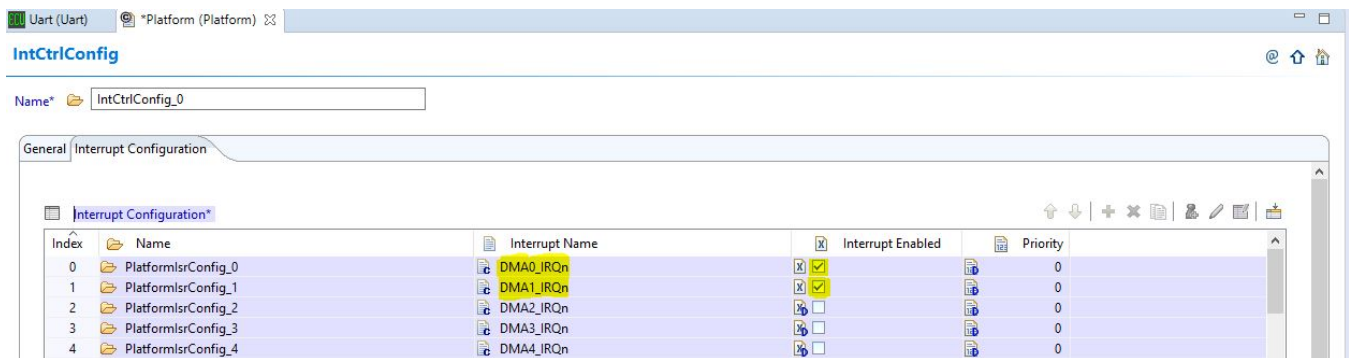


Figure 3.21 Enable DMA IRQ Configuration - Platform module in EB Tresos

Platform

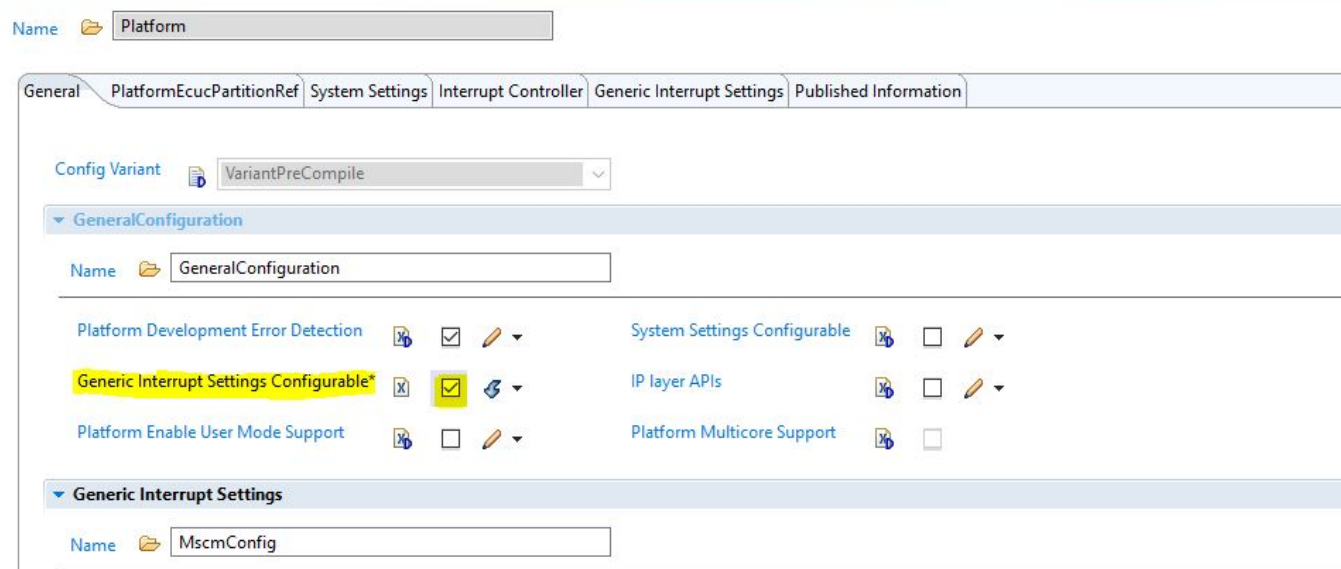


Figure 3.22 Enable Generic Interrupt Setting Configuration - Platform module in EB Tresos

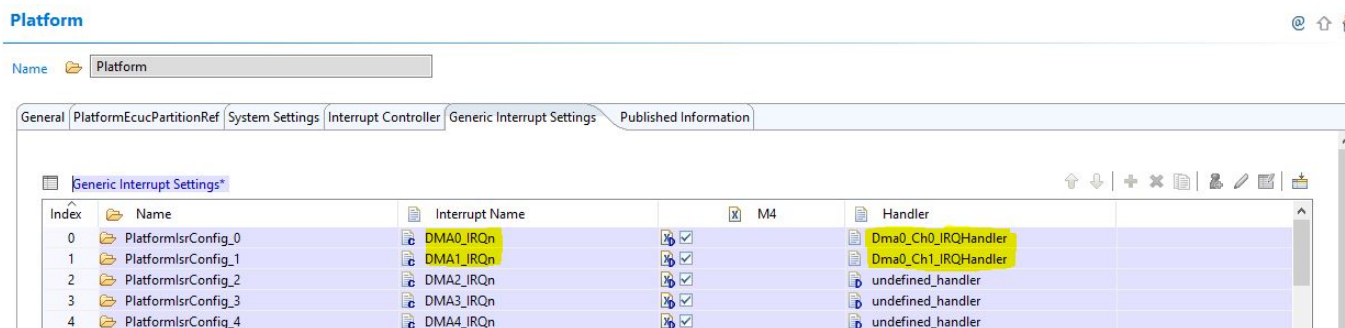


Figure 3.23 Fill In DMA IRQ Function Name - Platform module in EB Tresos

Note:

- About configuration of DMA Callback notification please refer chapter 6.3 Calls to Notification Functions, Callbacks, Callouts in Integration Manual document.
- Address of Buffer is used for Transmit and Receive data with DMA should be placed in UNSPECIFIED_N←O_CACHEABLE area please refer chapter 5.6 Data Cache Restrictions in Integration Manual document.

3.6.8 How to configure Idle Interrupt

In Idle Interrupt mode, for reception operation, the LPUART will be configured to start detecting an idle character from the previous stop bit (any data bits and stop bits count towards the idle character detection). And an idle characters that must be received before an idle line condition is detected. It only support for the LPUART.

For enabling this mode, you can check the UartTimeoutEnable for each channel that will be configured in this mode.

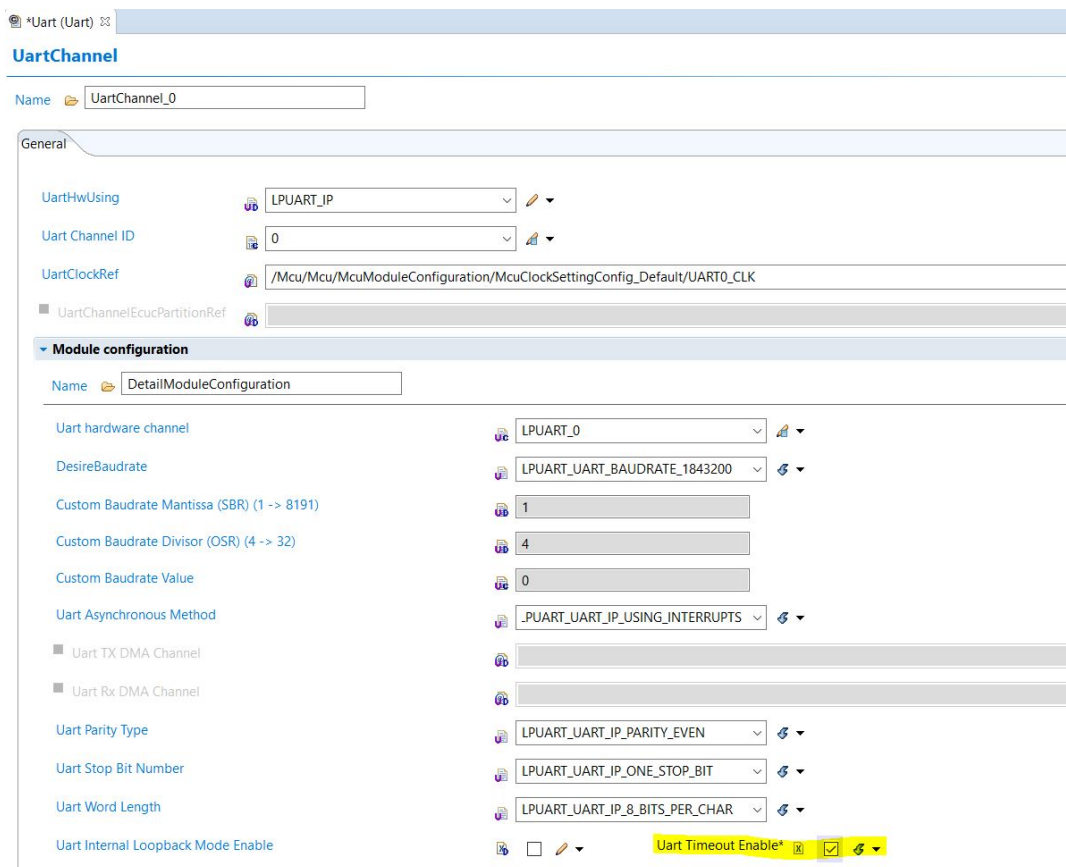


Figure 3.24 Idle interrupt mode internal architecture

Note:

- When the mode is enable for DMA, please add configuration for the interrupt handle to use.

3.7 Runtime errors

The driver generates the following DEM errors at runtime.

Function	Error Code	Condition triggering the error
Uart_SyncSend	UART_E_TIMEOUT	If an error occurs in transmit process, then transmitter can not send entire the message for a period of time longer than the configured timeout
Uart_SyncReceive	UART_E_TIMEOUT	If an error occurs in receive process or the receiver waits the message for a period of time longer than the configured timeout
Uart_Deinit	UART_E_DEINIT_FAILED	If one or more specific channels are in transmission progress, then Deinit channel will finish unsuccessfully

3.8 Symbolic Names Disclaimer

All containers having symbolicNameValue set to TRUE in the AUTOSAR schema will generate defines like: `#define <Mip>Conf_<Container_ShortName>_<Container_ID>`

For this reason it is forbidden to duplicate the names of such containers across the RTD configurations or to use names that may trigger other compile issues (e.g. match existing `#ifdefs` arguments).

3.9 Usage in non AUTOSAR context

For the non-ASR environments, both High-Level and IP layer interfaces are exposed and can be used by software units above. These may consist of either higher level NXP SW deliverables, like stacks and middleware, or the application code itself.

The goal of the product is to be versatile, easy to use and integrate with other SW environments, offering both standardized and optimized low-level interfaces.

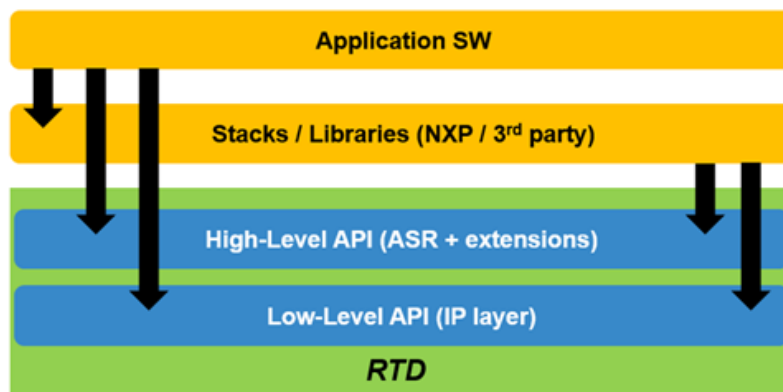


Figure 3.25 RTD Usage in non-ASR Context

In the non-AUTOSAR environment, there are no specific ECU states defined, but an external assumption for the application is that the correct sequence of initialization takes place prior to calling any runtime APIs.

The constraints considered for defining the APIs in the non-AUTOSAR environment are:

- Hardware resources used by the drivers must be properly initialized prior to runtime usage by calling the initialization function with configuration parameters generated by the configuration tools
- All dependencies must be initialized prior to the usage of a driver that depends on them; this translates to a general initialization sequence applicable for all applications, as follows:
 1. Initialization of the device clock tree so other periphery can be initialized

2. Configuration of the pinout so peripherals with external connections can communicate with the outside world
3. Configuration of the common resources (e.g. DMA channels, memory regions, access rights, etc) prior to the usage of any runtime functionality that has to benefit from this overall SoC configuration
4. Initialization of specific drivers
5. Runtime functionality
6. Deinitialization (inverse initialization order)

High-level and IP layer functionalities cannot be used simultaneously over the same hardware resource. The granularity of the resource must be defined specifically for each driver, but as a general rule the default granularity considered is the peripheral instance (e.g. Gpt high level APIs cannot be called interchangeably with Pit_Ip low level APIs over the same PIT hardware instance).

The diagram below shows a generic, high-level sequence of calls to RTD interfaces in the non-AUTOSAR scenario:

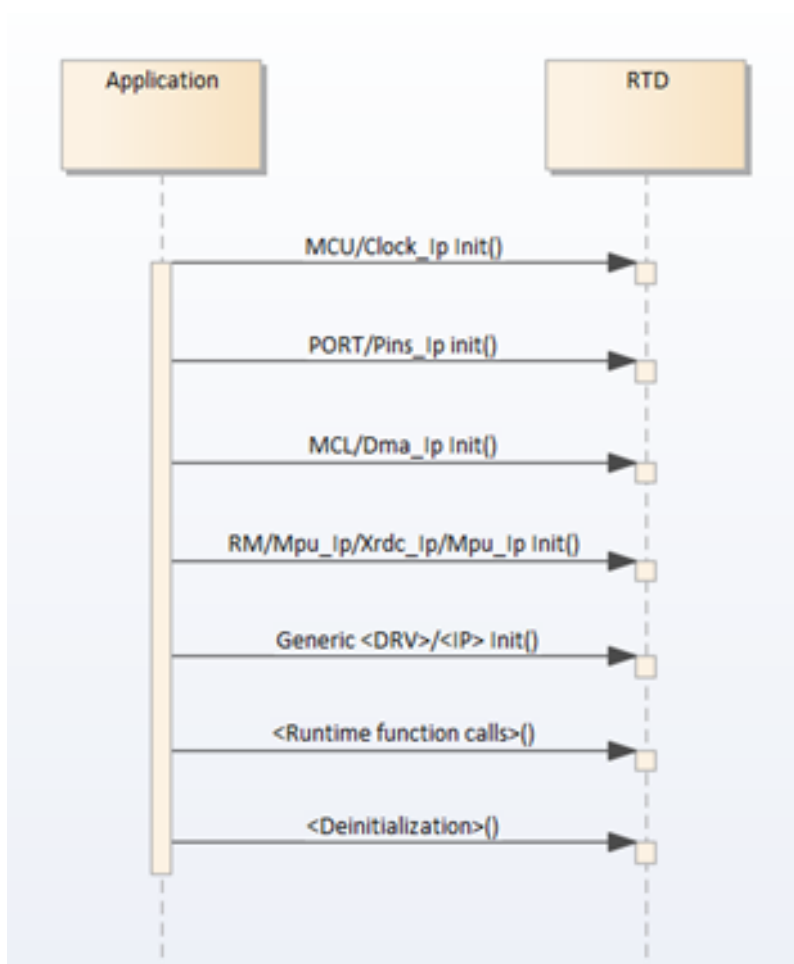


Figure 3.26 Non-AUTOSAR Sequence diagram

Chapter 4

Configuration Parameters

RTD provides configuration schema for the high level driver in EB Tresos proprietary format (**.xdm**), S32CT proprietary format (**.component**), but also in a standardized AUTOSAR format (**.epd**). For the low level driver configuration only the S32CT proprietary format (**.component**) is supported. The configuration data of the project can be saved in EB Tresos proprietary format (**.xdm**) and AUTOSAR standardized format (**.epc**) from EB Tresos and in S32CT proprietary format (**.mex**) and AUTOSAR standardized format (**.epc**) from S32 Design Studio. RTD provides templates for parsing the configuration data and generating configuration structures from it in XPath format, to be used with the EB Tresos generator and in JS format, to be used with the S32CT generator. Since the generation process is not standardized by AUTOSAR, only the two mentioned tools can be used with the provided templates. Using the same input configuration data, the same configuration structures will be obtained using any of the generators.

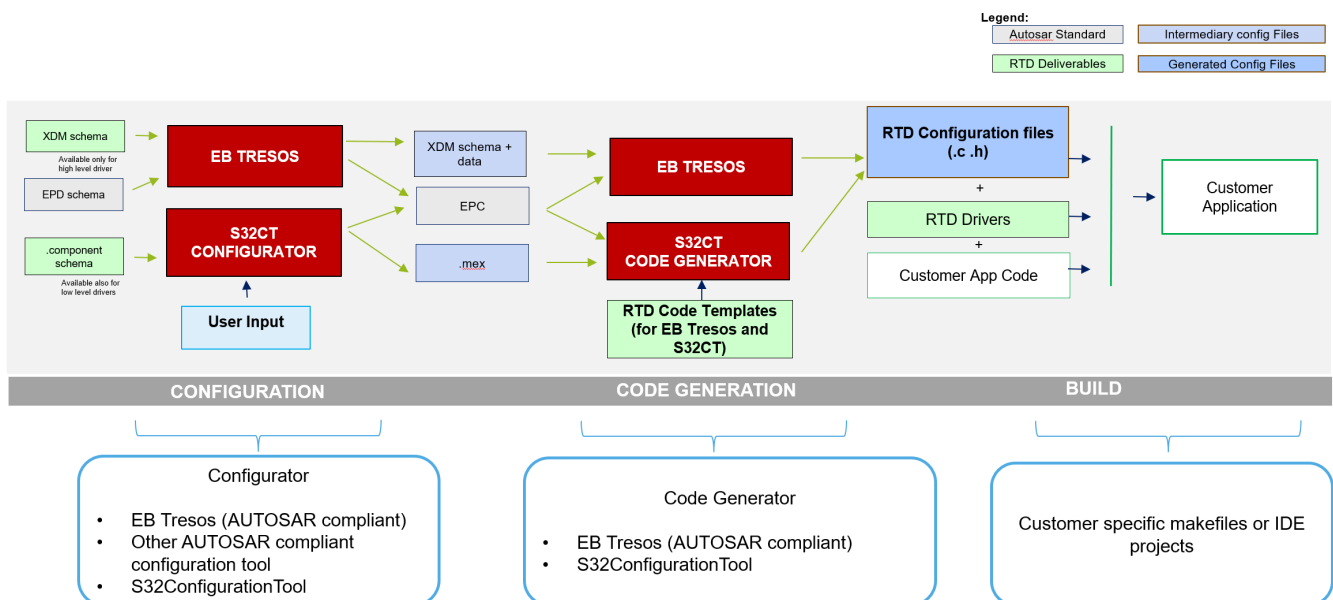


Figure 4.1 Configuration generation flow

- Uart
- LPUART_UART
- FLEXIO_UART

4.1 Uart

- Module [Uart](#)
 - Container [GeneralConfiguration](#)
 - * Parameter [UartDevErrorDetect](#)
 - * Parameter [DisableUartRuntimeErrorDetect](#)
 - * Parameter [UartMultipartitionSupport](#)
 - * Parameter [UartEnableUserModeSupport](#)
 - * Parameter [UartTimeoutMethod](#)
 - * Parameter [UartTimeoutDuration](#)
 - * Parameter [UartDmaEnable](#)
 - * Parameter [UartVersionInfoApi](#)
 - * Parameter [UartCallbackCapability](#)
 - * Parameter [UartCallback](#)
 - * Reference [UartEcucPartitionRef](#)
 - Container [UartGlobalConfig](#)
 - * Container [UartChannel](#)
 - Parameter [UartHwUsing](#)
 - Parameter [UartChannelId](#)
 - Reference [UartClockRef](#)
 - Reference [UartChannelEcucPartitionRef](#)
 - Container [DetailModuleConfiguration](#)
 - Parameter [UartHwChannel](#)
 - Parameter [DesireBaudrate](#)
 - Parameter [CustomBaudrateMantissa](#)
 - Parameter [CustomBaudrateDivisor](#)
 - Parameter [CustomBaudrateValue](#)
 - Parameter [UartInterruptDmaMethod](#)
 - Parameter [UartParityType](#)
 - Parameter [UartStopBitNumber](#)
 - Parameter [UartWordLength](#)
 - Parameter [UartInternalLoopbackEnable](#)
 - Parameter [UartTimeoutEnable](#)
 - Reference [UartDmaTxChannelRef](#)
 - Reference [UartDmaRxChannelRef](#)
 - Container [FlexioModuleConfiguration](#)
 - Parameter [FlexioUartInterruptDmaMethod](#)

- Parameter [DesireBaudrate](#)
- Parameter [CustomTimerDecrement](#)
- Parameter [CustomBaudrateDivider](#)
- Parameter [CustomBaudrateValue](#)
- Parameter [bitCount](#)
- Parameter [driverDirection](#)
- Reference [UartHwChannelRef](#)
- Reference [FlexioDmaChannelRef](#)
- Container [CommonPublishedInformation](#)
 - * Parameter [ArReleaseMajorVersion](#)
 - * Parameter [ArReleaseMinorVersion](#)
 - * Parameter [ArReleaseRevisionVersion](#)
 - * Parameter [ModuleId](#)
 - * Parameter [SwMajorVersion](#)
 - * Parameter [SwMinorVersion](#)
 - * Parameter [SwPatchVersion](#)
 - * Parameter [VendorApiInfix](#)
 - * Parameter [VendorId](#)

4.1.1 Module Uart

Configuration of the Universal asynchronous receiver-transmitter (Uart) module.

Included containers:

- [GeneralConfiguration](#)
- [UartGlobalConfig](#)
- [CommonPublishedInformation](#)

Property	Value
type	ECUC-MODULE-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD, VARIANT-PRE-COMPILE

4.1.2 Container GeneralConfiguration

GeneralConfiguration

This container contains the global configuration parameters of the Non-Autosar Uart driver.

Note: Implementation Specific Parameter.

Included subcontainers:

Configuration Parameters

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.3 Parameter UartDevErrorDetect

UartDevErrorDetect

Switches the Development Error Detection and Notification ON or OFF.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	true

4.1.4 Parameter DisableUartRuntimeErrorDetect

DisableUartRuntimeErrorDetect

Switches the Runtime Error Detection and Notification ON or OFF.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE

Property	Value
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.5 Parameter UartMultipartitionSupport

UartMultipartitionEnable

When this parameter is enabled, multi-partition feature will be used in Uart driver.

That means mapping the Uart driver to multiple ECUC partitions to make the module API available in this partition.

The Uart driver will operate as an independent instance in each of the partitions.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.6 Parameter UartEnableUserModeSupport

When this parameter is enabled, the MDL module will adapt to run from User Mode, with the following measures:

a) configuring REG_PROT for ABC1, ABC2 IPs so that the registers under protection can be accessed from user mode by setting UAA bit in REG_PROT_GCR to 1

b) using 'call trusted function' stubs for all internal function calls that access registers requiring supervisor mode.

c) other module specific measures

for more information, please see chapter 5.7 User Mode Support in IM

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE

Property	Value
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.7 Parameter UartTimeoutMethod

This parameter is used to select between different OsIf counter implementations. For additional details, please refer to the OsIf documentation.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	OSIF_COUNTER_DUMMY
literals	['OSIF_COUNTER_DUMMY', 'OSIF_COUNTER_SYSTEM', 'OSIF_COUNTER_CUSTOM']

4.1.8 Parameter UartTimeoutDuration

Specifies the maximum number of loops for blocking function until a timeout is raised in short term wait loops. If LinTimeoutMethod is OSIF_COUNTER_SYSTEM or OSIF_COUNTER_CUSTOM, UartTimeoutDuration is in microsecond value. If LinTimeoutMethod is OSIF_COUNTER_DUMMY, the UartTimeoutDuration is number of wait loop.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1000
max	4294967295
min	0

4.1.9 Parameter UartDmaEnable

UartDmaEnable

Check this in order to be able to use DMA in the Uart driver.

Leaving this unchecked will allow the Uart driver to compile with no dependencies from the Mcl driver.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.10 Parameter UartVersionInfoApi

UartVersionInfoApi

Switches the Uart_GetVersionInfo function ON or OFF.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.1.11 Parameter UartCallbackCapability

When this parameter is enabled, Callback feature will be used in Uart driver, with the following functions:

- a) completion of a reception or sending operation.*
- b) reporting communication errors.*

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.12 Parameter UartCallback

Callback for the rx full buffer event.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	NULL_PTR

4.1.13 Reference UartEcucPartitionRef

ECUC_Uart_00244.Maps the Uart driver to zero or multiple ECUC partitions to make the driver

API available in the according partition.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

Property	Value
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/EcuC/EcuPartitionCollection/EcuPartition

4.1.14 Container UartGlobalConfig

This container contains the global configuration parameter of the Uart driver. This container is a MultipleConfigurationContainer, i.e. this container and its sub-containers exit once per configuration set.

Included subcontainers:

- [UartChannel](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.15 Container UartChannel

This container contains the configuration (parameters) of the Uart Controller(s).

Note: "User should use unique names for naming the Uart channels across different UartGlobalConfig Sets."

Included subcontainers:

- [DetailModuleConfiguration](#)
- [FlexioModuleConfiguration](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	6
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.1.16 Parameter UartHwUsing

This parameter is user's desire Uart Hardware.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
default Value	LPUART_IP
literals	['LPUART_IP', 'FLEXIO_IP']

4.1.17 Parameter UartChannelId

Identifies the Uart channel.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	1
max	6
min	0

4.1.18 Reference UartClockRef

Reference to the Uart clock source configuration, which is set into the MCU driver configuration.

This clock source is used for configure Uart baudrate.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/Mcu/McuModuleConfiguration/McuClockSetting↔ Config/McuClockReferencePoint

4.1.19 Reference UartChannelEcucPartitionRef

Maps one single Uart channel to zero or one ECUC partitions.

The ECUC partition referenced is a subset of the ECUC partitions where the Uart driver is mapped to.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/EcuC/EcucPartitionCollection/EcucPartition

4.1.20 Container DetailModuleConfiguration

This container contains the configuration (parameters) of the Module Configuration.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.21 Parameter UartHwChannel

Selects the physical Lpuart Uart Channel.

Configuration Parameters

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	LPUART_0
literals	['LPUART_0', 'LPUART_1']

4.1.22 Parameter DesireBaudrate

This parameter is user's desire baudrate.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	LPUART_UART_BAUDRATE_9600
literals	['LPUART_UART_BAUDRATE_CUSTOM', 'LPUART_UART_BAUDRATE_1200', 'LPUART_UART_BAUDRATE_2400', 'LPUART_UART_BAUDRATE_4800', 'LPUART_UART_BAUDRATE_7200', 'LPUART_UART_BAUDRATE_9600', 'LPUART_UART_BAUDRATE_14400', 'LPUART_UART_BAUDRATE_19200', 'LPUART_UART_BAUDRATE_28800', 'LPUART_UART_BAUDRATE_38400', 'LPUART_UART_BAUDRATE_57600', 'LPUART_UART_BAUDRATE_115200', 'LPUART_UART_BAUDRATE_230400', 'LPUART_UART_BAUDRATE_460800', 'LPUART_UART_BAUDRATE_921600', 'LPUART_UART_BAUDRATE_1843200']

4.1.23 Parameter CustomBaudrateMantissa

Identifies the custom baudrate value.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	1
max	8191
min	1

4.1.24 Parameter CustomBaudrateDivisor

Identifies the custom baudrate value.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	4
max	32
min	4

4.1.25 Parameter CustomBaudrateValue

Identifies the custom baudrate value.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP

Configuration Parameters

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	9223372036854775807
min	-9223372036854775808

4.1.26 Parameter UartInteruptDmaMethod

Configures the mechanism used by the 'AsyncSend' or 'AsyncReceive' function (interrupts or DMA).

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	LPUART_UART_IP_USING_INTERRUPTS
literals	['LPUART_UART_IP_USING_INTERRUPTS', 'LPUART_UART_IP_U↵ SING_DMA']

4.1.27 Parameter UartParityType

Configures the type of parity to be used for UART bytes.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	LPUART_UART_IP_PARITY_DISABLED
literals	['LPUART_UART_IP_PARITY_DISABLED', 'LPUART_UART_IP_PA← RITY_EVEN', 'LPUART_UART_IP_PARITY_ODD']

4.1.28 Parameter UartStopBitNumber

Configures the number of stop bit to be used for UART frame.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	LPUART_UART_IP_ONE_STOP_BIT
literals	['LPUART_UART_IP_ONE_STOP_BIT', 'LPUART_UART_IP_TWO_← STOP_BIT']

4.1.29 Parameter UartWordLength

Configures the word length in UART mode.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE

Configuration Parameters

Property	Value
defaultValue	LPUART_UART_IP_8_BITS_PER_CHAR
literals	['LPUART_UART_IP_7_BITS_PER_CHAR', 'LPUART_UART_IP_8_BITS_PER_CHAR', 'LPUART_UART_IP_9_BITS_PER_CHAR', 'LPUART_UART_IP_10_BITS_PER_CHAR']

4.1.30 Parameter UartInternalLoopbackEnable

UartInternalLoopbackEnable

This check enables the internal loopback for the current channel.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.31 Parameter UartTimeoutEnable

UartTimeoutEnable

This check enables the Uart Timeout Interrupt for the current channel.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.32 Reference UartDmaTxChannelRef

Reference to the DMA TX channel, which is set in the Mcl driver configuration.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/Mcl/MclConfig/dmaLogicChannel_Type

4.1.33 Reference UartDmaRxChannelRef

Reference to the DMA Rx channel, which is set in the Mcl driver configuration.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/Mcl/MclConfig/dmaLogicChannel_Type

4.1.34 Container FlexioModuleConfiguration

This container contains the configuration (parameters) of the Flexio Uart Configuration.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	1

Configuration Parameters

Property	Value
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.35 Parameter FlexioUartInterruptDmaMethod

Configures the mechanism used by the 'AsyncSend' or 'AsyncReceive' function (interrupts or DMA).

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	FLEXIO_UART_IP_DRIVER_TYPE_INTERRUPTS
literals	['FLEXIO_UART_IP_DRIVER_TYPE_INTERRUPTS', 'FLEXIO_UART_IP_DRIVER_TYPE_DMA']

4.1.36 Parameter DesireBaudrate

This parameter is user's desire baudrate.

Note: Implementation Specific Parameter. In the case of baudrate is 1200, should use clocks less than 155MHz

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	FLEXIO_UART_BAUDRATE_9600

Property	Value
literals	['FLEXIO_UART_BAUDRATE_CUSTOM', 'FLEXIO_UART_BAUDRATE_1200', 'FLEXIO_UART_BAUDRATE_2400', 'FLEXIO_UART_BAUDRATE_4800', 'FLEXIO_UART_BAUDRATE_7200', 'FLEXIO_UART_BAUDRATE_9600', 'FLEXIO_UART_BAUDRATE_14400', 'FLEXIO_UART_BAUDRATE_19200', 'FLEXIO_UART_BAUDRATE_28800', 'FLEXIO_UART_BAUDRATE_38400', 'FLEXIO_UART_BAUDRATE_57600', 'FLEXIO_UART_BAUDRATE_115200', 'FLEXIO_UART_BAUDRATE_230400', 'FLEXIO_UART_BAUDRATE_460800', 'FLEXIO_UART_BAUDRATE_921600', 'FLEXIO_UART_BAUDRATE_1843200']

4.1.37 Parameter CustomTimerDecrement

This parameter is user's custom baudrate.

Note: Implementation Specific Parameter. In the case of baudrate is 1200, should use clocks less than 155MHz

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	FLEXIO_TIMER_DECREMENT_FXIO_CLK_SHIFT_TMR
literals	['FLEXIO_TIMER_DECREMENT_FXIO_CLK_SHIFT_TMR', 'FLEXIO_TIMER_DECREMENT_FXIO_CLK_DIV_16', 'FLEXIO_TIMER_DECREMENT_FXIO_CLK_DIV_256']

4.1.38 Parameter CustomBaudrateDivider

Identifies the custom baudrate value.

Note: The Custom Baudrate Divider should be even when the CMP bit is equal to (Divider/2 - 1).

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true

Property	Value
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	32
max	512
min	2

4.1.39 Parameter CustomBaudrateValue

Identifies the custom baudrate value.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	9223372036854775807
min	-9223372036854775808

4.1.40 Parameter bitCount

Configures the word length in UART mode.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	FLEXIO_UART_IP_8_BITS_PER_CHAR
literals	['FLEXIO_UART_IP_8_BITS_PER_CHAR']

4.1.41 Parameter driverDirection

Configures the mechanism used by the Send or Receive function (TX or RX).

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	FLEXIO_UART_IP_DIRECTION_TX
literals	['FLEXIO_UART_IP_DIRECTION_TX', 'FLEXIO_UART_IP_DIRECTION_RX']

4.1.42 Reference UartHwChannelRef

Uart Flexio Channel Reference

Reference to the Flexio Channel configure for the Request

Property	Value
type	ECUC-CHOICE-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destinations	['/TS_T40D2M30I0R0/Mcl/MclConfig/FlexioCommon/FlexioMclLogicChannels']

4.1.43 Reference FlexioDmaChannelRef

Reference to the DMA TX or RX channel, which is set in the Mcl driver configuration.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-REFERENCE-DEF

Configuration Parameters

Property	Value
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/Mcl/MclConfig/dmaLogicChannel_Type

4.1.44 Container CommonPublishedInformation

Common container, aggregated by all modules.

It contains published information about vendor and versions.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.45 Parameter ArReleaseMajorVersion

Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.1.46 Parameter ArReleaseMinorVersion

Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	7
max	7
min	7

4.1.47 Parameter ArReleaseRevisionVersion

Revision version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.48 Parameter ModuleId

Module ID of this module from Module List.

Note: Implementation Specific Parameter

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP

Configuration Parameters

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	255
max	255
min	255

4.1.49 Parameter SwMajorVersion

Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Note: Implementation Specific Parameter

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	3
max	3
min	3

4.1.50 Parameter SwMinorVersion

Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Note: Implementation Specific Parameter

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.51 Parameter SwPatchVersion

Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.

Note: Implementation Specific Parameter

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.52 Parameter VendorApiInfix

In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name.

This parameter is used to specify the vendor specific name. In total, the Implementation specific name is generated as follows:

<ModuleName>_<VendorId>_<VendorApiInfix>.

E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name Can_Write defined in the SWS will translate to Can_123_v11r456Write.

This parameter is mandatory for all modules with upper multiplicity >

1. It shall not be used for modules with upper multiplicity =1.

Note: Implementation Specific Parameter

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	

4.1.53 Parameter VendorId

Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Note: Implementation Specific Parameter

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	43
max	43
min	43

4.2 LPUART_UART

The low level driver component can only be used with S32CT Configurator in S32 Design Studio.

- Module [Lpuart_Uart](#)
 - Container [GeneralConfiguration](#)
 - * Parameter [UartDevErrorDetect](#)
 - * Parameter [UartTimeoutMethod](#)
 - * Parameter [UartTimeoutDuration](#)
 - * Parameter [UartDmaEnable](#)
 - * Parameter [UartCallbackCapability](#)
 - * Parameter [UartCallback](#)

- Container [UartGlobalConfig](#)
 - * Container [UartChannel](#)
 - Parameter [UartClockRef](#)
 - Container [DetailModuleConfiguration](#)

4.2.1 Module Lpuart_Uart

Configuration of the Lpuart_Uart (Uart Driver) module.

Included containers:

- [GeneralConfiguration](#)
- [UartGlobalConfig](#)

Property	Value
type	MODULE-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD;VARIANT-PRE-COMPILE

4.3 FLEXIO_UART

The low level driver component can only be used with S32CT Configurator in S32 Design Studio.

- Module [Flexio_Uart](#)
 - Container [GeneralConfiguration](#)
 - * Parameter [UartDevErrorDetect](#)
 - * Parameter [UartTimeoutMethod](#)
 - * Parameter [UartTimeoutDuration](#)
 - * Parameter [UartDmaEnable](#)
 - * Parameter [UartCallbackCapability](#)
 - * Parameter [UartCallback](#)
 - Container [UartGlobalConfig](#)
 - * Container [UartChannel](#)
 - Parameter [UartClockRef](#)
 - Container [FlexioModuleConfiguration](#)

4.3.1 Module Flexio_Uart

Configuration of the Flexio_Uart (Uart Driver) module.

Included containers:

- [GeneralConfiguration](#)

Configuration Parameters

- [UartGlobalConfig](#)

Property	Value
type	MODULE-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD;VARIANT-PRE-COMPILE



Chapter 5

Module Index

5.1 Software Specification

Here is a list of all modules:

UART Driver	58
Flexio UART IPL	68
Lpuart UART IPL	72

Chapter 6

Module Documentation

6.1 UART Driver

6.1.1 Detailed Description

Data Structures

- struct [Uart_ChannelConfigType](#)
Uart channel configuration type structure. [More...](#)
- struct [Uart_ConfigType](#)
Uart driver configuration type structure. [More...](#)

Enum Reference

- enum [Uart_DrvStatusType](#)
Driver initialization status.
- enum [Uart_DataDirectionType](#)
The type operation of an Uart channel.
- enum [Uart_StatusType](#)
Uart operation status type.

Function Reference

- void [Uart_Init](#) (const [Uart_ConfigType](#) *Config)
Initializes the UART module.
- void [Uart_Deinit](#) (void)
De-initializes the UART module.
- Std_ReturnType [Uart_SetBaudrate](#) (uint8 Channel, Uart_BaudrateType Baudrate)
Configures the baud rate for the serial communication.
- Std_ReturnType [Uart_GetBaudrate](#) (uint8 Channel, uint32 *Baudrate)
Retrieves the baud rate which is currently set for the serial communication.
- void [Uart_SetBuffer](#) (uint8 Channel, uint8 *Buffer, uint32 BufferSize, [Uart_DataDirectionType](#) Direction)
Configures a new buffer for continuous transfers.
- Std_ReturnType [Uart_SyncSend](#) (uint8 Channel, const uint8 *Buffer, uint32 BufferSize, uint32 Timeout)

Starts a synchronous transfer of bytes.

- Std_ReturnType [Uart_SyncReceive](#) (uint8 Channel, uint8 *Buffer, uint32 BufferSize, uint32 Timeout)

Starts a synchronous reception of bytes.

- Std_ReturnType [Uart_Abort](#) (uint8 Channel, [Uart_DataDirectionType](#) TransmissionType)

Aborts an on-going transfer.

- Std_ReturnType [Uart_AsyncSend](#) (uint8 Channel, const uint8 *Buffer, uint32 BufferSize)

Starts an asynchronous transfer(send) of bytes.

- Std_ReturnType [Uart_AsyncReceive](#) (uint8 Channel, uint8 *Buffer, uint32 BufferSize)

Starts an asynchronous transfer(receive) of bytes.

- [Uart_StatusType](#) [Uart_GetStatus](#) (uint8 Channel, uint32 *BytesTransferred, [Uart_DataDirectionType](#) TransferType)

Returns the status of the previous transfer.

6.1.2 Data Structure Documentation

6.1.2.1 struct Uart_ChannelConfigType

Uart channel configuration type structure.

This is the type of the external data structure containing the overall initialization data for one Uart Channel. A pointer to such a structure is provided to the Uart channel initialization routine for configuration of the Uart hardware channel.

Definition at line 321 of file Uart_Types.h.

Data Fields

uint8	UartChannelId	Uart channel configured
uint32	ChannelClockFrequency	The clock frequency configured on the given channel
const Uart_Ipw_HwConfigType *	UartChannelConfig	Pointer to a lower level channel configuration

6.1.2.2 struct Uart_ConfigType

Uart driver configuration type structure.

This is the type of the pointer to the external data Uart Channels. A pointer of such a structure is provided to the Uart driver initialization routine for configuration of the Uart hardware channel.

Definition at line 343 of file Uart_Types.h.

Data Fields

const Uart_ChannelConfigType *	Configs[UART_CH_MAX_CONFIG]	Hardware channel. Constant pointer of the constant external data structure containing the overall initialization data for all the configured Uart Channels.
--	-----------------------------	---

6.1.3 Enum Reference

6.1.3.1 Uart_DrvStatusType

enum `Uart_DrvStatusType`

Driver initialization status.

This enum contains the values for the driver initialization status.

Enumerator

UART_DRV_UNINIT	Driver not initialized.
UART_DRV_INIT	Driver ready.

Definition at line 269 of file `Uart_Types.h`.

6.1.3.2 Uart_DataDirectionType

enum `Uart_DataDirectionType`

The type operation of an Uart channel.

Enumerator

UART_SEND	The sending operation.
UART_RECEIVE	The receiving operation.

Definition at line 281 of file `Uart_Types.h`.

6.1.3.3 Uart_StatusType

enum `Uart_StatusType`

Uart operation status type.

Enumerator

UART_STATUS_NO_ERROR	Uart operation is successfull
UART_STATUS_OPERATION_ONGOING	Uart operation on going
UART_STATUS_ABORTED	Uart operation aborted
UART_STATUS_FRAMING_ERROR	Uart framing error
UART_STATUS_RX_OVERRUN_ERROR	Uart overrun error
UART_STATUS_PARITY_ERROR	Uart parity error

Enumerator

UART_STATUS_TIMEOUT	Uart operation has timeout
UART_STATUS_NOISE_ERROR	Uart noise error
UART_STATUS_DMA_ERROR	Uart Dma error
UART_STATUS_BREAK_ERROR	Uart break error

Definition at line 290 of file Uart_Types.h.

6.1.4 Function Reference

6.1.4.1 Uart_Init()

```
void Uart_Init (
    const Uart_ConfigType * Config )
```

Initializes the UART module.

This function performs software initialization of UART driver. It shall configure the Uart hardware peripheral for each channel.

Parameters

in	<i>Config</i>	- Pointer to UART driver configuration set.
----	---------------	---

Returns

void

Precondition

-

6.1.4.2 Uart_Deinit()

```
void Uart_Deinit (
    void )
```

De-initializes the UART module.

This function performs software de-initialization of UART driver.

Parameters

-	
---	--

Returns

void

Precondition

-

6.1.4.3 Uart_SetBaudrate()

```
Std_ReturnType Uart_SetBaudrate (
    uint8 Channel,
    Uart_BaudrateType Baudrate )
```

Configures the baud rate for the serial communication.

This function performs the setting of the communication baudrate provided in the parameter.

Parameters

in	<i>Channel</i>	- Uart channel to be addressed.
in	<i>Baudrate</i>	- Baudrate value to be set.

Returns

Std_ReturnType.

Return values

<i>E_NOT_OK</i>	If the Uart Channel is not valid or Uart driver is not initialized or a transfer is on-going or wrong partition is addressed.
<i>E_OK</i>	Successfull baudrate configuration.

Precondition

Uart_Init function must be called before this API.

6.1.4.4 Uart_GetBaudrate()

```
Std_ReturnType Uart_GetBaudrate (
    uint8 Channel,
    uint32 * Baudrate )
```

Retrieves the baud rate which is currently set for the serial communication.

This function returns via the second parameter the current serial baudrate.

Parameters

in	<i>Channel</i>	- Uart channel to be addressed.
out	<i>Baudrate</i>	- Pointer to a memory location where the baudrate value is returned.

Returns

Std_ReturnType.

Return values

<i>E_NOT_OK</i>	If the Uart Channel is not valid or Uart driver is not initialized or a transfer is on-going or wrong partition is addressed or a NULL_PTR pointer has been provided
<i>E_OK</i>	Successfull baudrate retrieval.

Precondition

Uart_Init function must be called before this API. Otherwise a random value can be returned.

6.1.4.5 Uart_SetBuffer()

```
void Uart_SetBuffer (
    uint8 Channel,
    uint8 * Buffer,
    uint32 BufferSize,
    Uart_DataDirectionType Direction )
```

Configures a new buffer for continuous transfers.

This function can be called inside a notification callback and offers the possibility to change the buffer in order to assure a continuous asynchronous transfer.

Parameters

in	<i>Channel</i>	- Uart channel to be addressed.
in	<i>Buffer</i>	- The new buffer provided.
in	<i>BufferSize</i>	- The size of the new buffer.
in	<i>Direction</i>	- This parameter indicates for which type of transmission is the buffer set. Its values are UART_SEND for setting a buffer when the previous buffer is empty and there are more bytes to send and UART_RECEIVE to set a new buffer when the previous buffer is full with received buffer and there is more data to be received.

Returns

void.

Precondition

Uart_Init function must be called before this API.

6.1.4.6 Uart_SyncSend()

```
Std_ReturnType Uart_SyncSend (
    uint8 Channel,
    const uint8 * Buffer,
    uint32 BufferSize,
    uint32 Timeout )
```

Starts a synchronous transfer of bytes.

This function starts sending a number of bytes in a synchronous manner.

Module Documentation

Parameters

in	<i>Channel</i>	- Uart channel to be addressed.
in	<i>Buffer</i>	- The buffer which contains the bytes to be sent.
in	<i>BufferSize</i>	- The Buffer size.
in	<i>Timeout</i>	- The timeout in us.

Returns

Std_ReturnType.

Return values

<i>E_NOT_OK</i>	If the Uart Channel is not valid or Uart driver is not initialized or Buffer is a NULL_PTR or BufferSize is 0, meaning no space has been allocated for the buffer or a wrong partition has been accessed or a transfer is already on going on the requested channel or timeout occurred.
<i>E_OK</i>	Successful transfer.

Precondition

Uart_Init function must be called before this API.

6.1.4.7 Uart_SyncReceive()

```
Std_ReturnType Uart_SyncReceive (  
    uint8 Channel,  
    uint8 * Buffer,  
    uint32 BufferSize,  
    uint32 Timeout )
```

Starts a synchronous reception of bytes.

This function starts receiving a number of bytes in a synchronous manner.

Parameters

in	<i>Channel</i>	- Uart channel to be addressed.
in	<i>Buffer</i>	- The buffer where the bytes will be located.
in	<i>BufferSize</i>	- The Buffer size.
in	<i>Timeout</i>	- The timeout in us.

Returns

Std_ReturnType.

Return values

<i>E_NOT_OK</i>	If the Uart Channel is not valid or Uart driver is not initialized or Buffer is a NULL_PTR or BufferSize is 0, meaning no space has been allocated for the buffer or a wrong partition has been accessed or a reception is already on going on the requested channel or timeout occurred.
<i>E_OK</i>	Successful reception.

Precondition

Uart_Init function must be called before this API.

6.1.4.8 Uart_Abort()

```
Std_ReturnType Uart_Abort (
    uint8 Channel,
    Uart_DataDirectionType TransmissionType )
```

Aborts an on-going transfer.

This function aborts either a reception or a transmission depending on the last parameter.

Parameters

in	<i>Channel</i>	- Uart channel to be addressed.
in	<i>TransmissionType</i>	- Type of the transfer to be aborted. It can be either UART_SEND or UART_RECEIVE.

Returns

Std_ReturnType.

Return values

<i>E_NOT_OK</i>	If the Uart Channel is not valid or Uart driver is not initialized or a wrong partition has been accessed
<i>E_OK</i>	Successful transfer aborted or in case no transfer was on going.

Precondition

Uart_Init function must be called before this API.

6.1.4.9 Uart_AsyncSend()

```
Std_ReturnType Uart_AsyncSend (
    uint8 Channel,
    const uint8 * Buffer,
    uint32 BufferSize )
```

Starts an asynchronous transfer(send) of bytes.

This function starts sending a number of bytes in an asynchronous manner. The transfer can be performed using either DMA or interrupts depending on the transfer type configured on the addressed channel.

Parameters

in	<i>Channel</i>	- Uart channel to be addressed.
in	<i>Buffer</i>	- The buffer where the data to be sent is located.
in	<i>BufferSize</i>	- The Buffer size.

Module Documentation

Returns

Std_ReturnType.

Return values

<i>E_NOT_OK</i>	If the Uart Channel is not valid or Uart driver is not initialized or Buffer is a NULL_PTR or BufferSize is 0, meaning no space has been allocated for the buffer or a wrong partition has been accessed or a transfer(send) is already on going on the requested channel.
<i>E_OK</i>	The transfer(send) started successfully.

Precondition

Uart_Init function must be called before this API.

6.1.4.10 Uart_AsyncReceive()

```
Std_ReturnType Uart_AsyncReceive (  
    uint8 Channel,  
    uint8 * Buffer,  
    uint32 BufferSize )
```

Starts an asynchronous transfer(receive) of bytes.

This function starts receiving a number of bytes in an asynchronous manner. The transfer can be performed using either DMA or interrupts depending on the transfer type configured on the addressed channel.

Parameters

in	<i>Channel</i>	- Uart channel to be addressed.
in	<i>Buffer</i>	- The buffer where the data to be received will located.
in	<i>BufferSize</i>	- The Buffer size.

Returns

Std_ReturnType.

Return values

<i>E_NOT_OK</i>	If the Uart Channel is not valid or Uart driver is not initialized or Buffer is a NULL_PTR or BufferSize is 0, meaning no space has been allocated for the buffer or a wrong partition has been accessed or a transfer(receive) is already on going on the requested channel.
<i>E_OK</i>	The transfer(receive) started successfully.

Precondition

Uart_Init function must be called before this API.

6.1.4.11 Uart_GetStatus()

```
Uart_StatusType Uart_GetStatus (  
    uint8 Channel,
```

```
uint32 * BytesTransferred,
Uart_DataDirectionType TransferType )
```

Returns the status of the previous transfer.

This function returns the status of the previous transfer. If there is a transfer in progress, this function will also get the number of remaining bytes at the time the function was called.

Parameters

in	<i>Channel</i>	- Uart channel to be addressed.
out	<i>BytesTransferred</i>	- A pointer where the number of remaining bytes will be written.
in	<i>TransferType</i>	- The type of transfer in discussion (UART_SEND or UART_RECEIVE).

Returns

Uart_StatusType.

Return values

<i>UART_STATUS_NO_ERROR</i>	- Operation has ended successfully.
<i>UART_STATUS_FRAMING_ERROR</i>	- Operation has had a framing error. This status is returned only if the TransferType parameter is RECEIVE.
<i>UART_STATUS_RX_OVERRUN_ERROR</i>	- Operation has had an overrun error. This status is returned only if the TransferType parameter is RECEIVE.
<i>UART_STATUS_PARITY_ERROR</i>	- Operation has had a parity error. This status is returned only if the TransferType parameter is RECEIVE.
<i>UART_STATUS_OPERATION_ONGOING</i>	- Operation has not finished at the moment of function call.
<i>UART_STATUS_ABORTED</i>	- Operation has been aborted.
<i>UART_STATUS_TIMEOUT</i>	- Operation has had timeout in synchronous transfer functions with timeout value pass in Timeout parameter or functions that use the loop function whose execution time exceeds the timeout value configured through the UI

Precondition

Uart_Init function must be called before this API.

6.2 Flexio UART IPL

6.2.1 Detailed Description

Data Structures

- struct [Flexio_Uart_Ip_UserConfigType](#)
Driver configuration structure. [More...](#)

Macros

- `#define FLEXIO_HW_INSTANCE`

Enum Reference

- enum [Flexio_Uart_IP_DriverType](#)
Driver type: Interrupts/DMA Implements : Flexio_Uart_IP_DriverType_Class.
- enum [Flexio_Uart_Ip_DirectionType](#)
flexio_uart driver direction (tx or rx)
- enum [Flexio_Uart_Ip_EventType](#)
Define the enum of the Events which can trigger UART callback.
- enum [Flexio_Uart_Ip_StatusType](#)
- enum [Flexio_Uart_Ip_BaudrateType](#)
Define the enum of the baudrate values that should be set for Uart.
- enum [Flexio_Uart_Ip_TimerDecrementType](#)
FLEXIO Timer decrement options.
- enum [Flexio_Uart_Ip_BitCountPerCharType](#)
FLEXIO number of bits in a character.

Variables

- [Flexio_Uart_Ip_StateStructureType](#)
Driver internal context structure.

6.2.2 Data Structure Documentation

6.2.2.1 struct Flexio_Uart_Ip_UserConfigType

Driver configuration structure.

This structure is used to provide configuration parameters for the flexio_uart driver at initialization time. Implements : [Flexio_Uart_Ip_UserConfigType_Class](#)

Definition at line 245 of file [Flexio_Uart_Ip_Types.h](#).

Data Fields

Flexio_Uart_IP_DriverType	uint32	Channel	Flexio Uart Channel has been configured. Note that Make sure the Channel is used in all API corresponds to this parameter.
		DriverType	Driver type: Interrupts/DMA.
	uint16	Divider	Baudrate divider.

Data Fields

Flexio_Uart_Ip_TimerDecrementType	TimerDec	The source of the Timer decrement and the source of the Shift clock.
uint32	BaudRate	Baud rate in hertz.
Flexio_Uart_Ip_BitCountPerCharType	BitCount	Number of bits per word.
Flexio_Uart_Ip_DirectionType	Direction	Driver direction: Tx or Rx.
uint8	DataPin	Flexio pin to use as Tx or Rx pin.
Flexio_Uart_Ip_CallbackType	Callback	User callback function. Note that this function will be called from the interrupt service routine, so its execution time should be as small as possible. It can be NULL_PTR if it is not needed.
void *	CallbackParam	Parameter for the callback function.
Flexio_Uart_Ip_StateStructureType *	StateStruct	

6.2.3 Macro Definition Documentation

6.2.3.1 FLEXIO_HW_INSTANCE

```
#define FLEXIO_HW_INSTANCE
```

FLEXIO Instance support .

Definition at line 94 of file Flexio_Uart_Ip_Types.h.

6.2.4 Enum Reference

6.2.4.1 Flexio_Uart_IP_DriverType

```
enum Flexio_Uart_IP_DriverType
```

Driver type: Interrupts/DMA Implements : Flexio_Uart_IP_DriverType_Class.

Enumerator

FLEXIO_UART_IP_DRIVER_TYPE_INTERRUPTS	Driver uses interrupts for data transfers
FLEXIO_UART_IP_DRIVER_TYPE_DMA	Driver uses DMA for data transfers

Definition at line 102 of file Flexio_Uart_Ip_Types.h.

6.2.4.2 Flexio_Uart_Ip_DirectionType

```
enum Flexio_Uart_Ip_DirectionType
```

flexio_uart driver direction (tx or rx)

This structure describes the direction configuration options for the flexio_uart driver. Implements : flexio_uart_↔ driver_direction_t_Class

Enumerator

FLEXIO_UART_IP_DIRECTION_TX	Tx UART driver
FLEXIO_UART_IP_DIRECTION_RX	Rx UART driver

Definition at line 114 of file Flexio_Uart_Ip_Types.h.

6.2.4.3 Flexio_Uart_Ip_EventType

enum `Flexio_Uart_Ip_EventType`

Define the enum of the Events which can trigger UART callback.

This enum should include the Events for all platforms

Enumerator

FLEXIO_UART_IP_EVENT_RX_FULL	Rx buffer is full.
FLEXIO_UART_IP_EVENT_TX_EMPTY	Tx buffer is empty.
FLEXIO_UART_IP_EVENT_END_TRANSFER	The current transfer is ending.
FLEXIO_UART_IP_EVENT_ERROR	An error occurred during transfer.

Definition at line 127 of file Flexio_Uart_Ip_Types.h.

6.2.4.4 Flexio_Uart_Ip_StatusType

enum `Flexio_Uart_Ip_StatusType`

Implements : Driver status type.

Enumerator

FLEXIO_UART_IP_STATUS_SUCCESS	Operation has been successfully
FLEXIO_UART_IP_STATUS_ERROR	Operation has had error
FLEXIO_UART_IP_STATUS_BUSY	Function is called during an on-going transfer
FLEXIO_UART_IP_STATUS_DMA_ERROR	Operation has had DMA error
FLEXIO_UART_IP_STATUS_TX_UNDERRUN	TX underrun error
FLEXIO_UART_IP_STATUS_RX_OVERRUN	RX overrun error
FLEXIO_UART_IP_STATUS_ABORTED	A transfer was aborted

Definition at line 138 of file Flexio_Uart_Ip_Types.h.

6.2.4.5 Flexio_Uart_Ip_BaudrateType

enum `Flexio_Uart_Ip_BaudrateType`

Define the enum of the baudrate values that should be set for Uart.

Definition at line 156 of file `Flexio_Uart_Ip_Types.h`.

6.2.4.6 Flexio_Uart_Ip_TimerDecrementType

enum `Flexio_Uart_Ip_TimerDecrementType`

FLEXIO Timer decrement options.

Enumerator

<code>FLEXIO_TIMER_DECREMENT_FXIO_CLK_↔ SHIFT_TMR</code>	Decrement counter on FlexIO clock, Shift clock equals Timer output.
<code>FLEXIO_TIMER_DECREMENT_FXIO_CLK_↔ DIV_16</code>	Decrement counter on FlexIO clock divided by 16, Shift clock equals Timer output.
<code>FLEXIO_TIMER_DECREMENT_FXIO_CLK_↔ DIV_256</code>	Decrement counter on FlexIO clock divided by 256, Shift clock equals Timer output.

Definition at line 180 of file `Flexio_Uart_Ip_Types.h`.

6.2.4.7 Flexio_Uart_Ip_BitCountPerCharType

enum `Flexio_Uart_Ip_BitCountPerCharType`

FLEXIO number of bits in a character.

Enumerator

<code>FLEXIO_UART_IP_8_BITS_PER_CHAR</code>	8-bit data characters
---	-----------------------

Definition at line 191 of file `Flexio_Uart_Ip_Types.h`.

6.2.5 Variable Documentation

6.2.5.1 Flexio_Uart_Ip_StateStructureType

`Flexio_Uart_Ip_StateStructureType`

Driver internal context structure.

This structure is used by the `flexio_uart` driver for its internal logic. It must be provided by the application through the `Flexio_Uart_Ip_Init()` function, then it cannot be freed until the driver is de-initialized using `Flexio_Uart_↔
Ip_Deinit()`. The application should make no assumptions about the content of this structure.

Definition at line 237 of file `Flexio_Uart_Ip_Types.h`.

6.3 Lpuart UART IPL

6.3.1 Detailed Description

Data Structures

- struct [Lpuart_Uart_Ip_StateStructureType](#)
Runtime of the LPUART driver. [More...](#)
- struct [Lpuart_Uart_Ip_UserConfigType](#)
LPUART configuration structure. [More...](#)

Types Reference

- typedef void(* [Lpuart_Uart_Ip_CallbackType](#)) (const uint8 HwInstance, const [Lpuart_Uart_Ip_EventType](#) Event, const void *UserData)
Callback for all peripherals which support UART features.

Enum Reference

- enum [Lpuart_Uart_Ip_TransferType](#)
Type of UART transfer (based on interrupts or DMA).
- enum [Lpuart_Uart_Ip_StatusType](#)
Driver status type.
- enum [Lpuart_Uart_Ip_EventType](#)
Define the enum of the Events which can trigger UART callback.
- enum [Lpuart_Uart_Ip_BaudrateType](#)
Define the enum of the baudrate values that should be set for Uart.

Function Reference

- void [Lpuart_Uart_Ip_Init](#) (const uint8 Instance, const [Lpuart_Uart_Ip_UserConfigType](#) *UserConfig)
Initializes an LPUART operation instance.
- [Lpuart_Uart_Ip_StatusType](#) [Lpuart_Uart_Ip_Deinit](#) (const uint8 Instance)
Shuts down the LPUART by disabling interrupts and transmitter/receiver.
- [Lpuart_Uart_Ip_StatusType](#) [Lpuart_Uart_Ip_SyncSend](#) (const uint8 Instance, const uint8 *TxBuff, const uint32 TxSize, const uint32 Timeout)
Send out multiple bytes of data using polling method.
- [Lpuart_Uart_Ip_StatusType](#) [Lpuart_Uart_Ip_AsyncSend](#) (const uint8 Instance, const uint8 *TxBuff, const uint32 TxSize)
Sends data out through the LPUART module using a non-blocking method. This enables an a-sync method for transmitting data. When used with a non-blocking receive, the LPUART can perform a full duplex operation. Non-blocking means that the function returns immediately. The application has to get the transmit status to know when the transmit is complete.
- [Lpuart_Uart_Ip_StatusType](#) [Lpuart_Uart_Ip_GetTransmitStatus](#) (const uint8 Instance, uint32 *Bytes←Remaining)
Returns whether the previous transmit is complete.
- [Lpuart_Uart_Ip_StatusType](#) [Lpuart_Uart_Ip_AbortSendingData](#) (const uint8 Instance)
Terminates a non-blocking transmission early.

- [Lpuart_Uart_Ip_StatusType Lpuart_Uart_Ip_SyncReceive](#) (const uint8 Instance, uint8 *RxBuff, const uint32 RxSize, const uint32 Timeout)

Receive multiple bytes of data using polling method.

- [Lpuart_Uart_Ip_StatusType Lpuart_Uart_Ip_AsyncReceive](#) (const uint8 Instance, uint8 *RxBuff, const uint32 RxSize)

Gets data from the LPUART module by using a non-blocking method. This enables an a-sync method for receiving data. When used with a non-blocking transmission, the LPUART can perform a full duplex operation. Non-blocking means that the function returns immediately. The application has to get the receive status to know when the receive is complete.

- [Lpuart_Uart_Ip_StatusType Lpuart_Uart_Ip_GetReceiveStatus](#) (const uint8 Instance, uint32 *Bytes←Remaining)

Returns whether the previous receive is complete.

- [Lpuart_Uart_Ip_StatusType Lpuart_Uart_Ip_AbortReceivingData](#) (const uint8 Instance)

Terminates a non-blocking receive early.

- [Lpuart_Uart_Ip_StatusType Lpuart_Uart_Ip_SetBaudRate](#) (const uint8 Instance, const [Lpuart_Uart_Ip_BaudrateType](#) DesiredBaudrate, const uint32 ClockFrequency)

Configures the LPUART baud rate.

- void [Lpuart_Uart_Ip_GetBaudRate](#) (const uint8 Instance, uint32 *ConfiguredBaudRate)

Returns the LPUART baud rate.

- void [Lpuart_Uart_Ip_SetTxBuffer](#) (const uint8 Instance, const uint8 *TxBuff, const uint32 TxSize)

Sets the internal driver reference to the tx buffer.

- void [Lpuart_Uart_Ip_SetRxBuffer](#) (const uint8 Instance, uint8 *RxBuff, const uint32 RxSize)

Sets the internal driver reference to the rx buffer.

6.3.2 Data Structure Documentation

6.3.2.1 struct Lpuart_Uart_Ip_StateStructureType

Runtime of the LPUART driver.

Note that the caller provides memory for the driver structures during initialization because the driver does not statically allocate memory.

Implements : [Lpuart_Uart_Ip_StateStructureType](#)

Definition at line 197 of file Lpuart_Uart_Ip_Types.h.

Data Fields

uint32	BaudRate	Variable that indicates if structure belongs to an instance already initialized.
const uint8 *	TxBuff	The buffer of data being sent.
uint8 *	RxBuff	The buffer of received data.
volatile uint32	TxSize	The remaining number of bytes to be transmitted.
volatile uint32	RxSize	The remaining number of bytes to be received.
volatile boolean	IsTxBusy	True if there is an active transmit.
volatile boolean	IsRxBusy	True if there is an active receive.
volatile Lpuart_Uart_Ip_StatusType	TransmitStatus	Status of last driver transmit operation.
volatile Lpuart_Uart_Ip_StatusType	ReceiveStatus	Status of last driver receive operation.

6.3.2.2 struct Lpuart_Uart_Ip_UserConfigType

LPUART configuration structure.

Implements : [Lpuart_Uart_Ip_UserConfigType](#)

Definition at line 215 of file Lpuart_Uart_Ip_Types.h.

Data Fields

uint32	BaudRate	Baudrate value.
uint32	BaudRateDivisor	Baud clock divisor.
uint8	BaudOverSamplingRatio	Over sampling ratio.
Lpuart_Uart_Ip_ParityModeType	ParityMode	Parity mode, disabled (default), even, odd.
Lpuart_Uart_Ip_StopBitCountType	StopBitsCount	Number of stop bits, 1 stop bit (default) or 2 stop bits.
Lpuart_Uart_Ip_BitCountPerCharType	BitCountPerChar	Number of bits in a character (8-default, 9 or 10); for 9/10 bits chars, users must provide appropriate buffers to the send/receive functions (bits 8/9 in subsequent bytes); for DMA transmission only 8-bit char is supported.
Lpuart_Uart_Ip_TransferType	TransferType	@briefType of LPUART transfer (interrupt/dma based)
Lpuart_Uart_Ip_CallbackType	Callback	Callback to invoke for handle uart event.
void *	CallbackParam	User callback parameter pointer.
Lpuart_Uart_Ip_StateStructureType *	StateStruct	

6.3.3 Types Reference

6.3.3.1 Lpuart_Uart_Ip_CallbackType

```
typedef void(* Lpuart_Uart_Ip_CallbackType) (const uint8 HwInstance, const Lpuart\_Uart\_Ip\_EventType Event, const void *UserData)
```

Callback for all peripherals which support UART features.

Definition at line 184 of file Lpuart_Uart_Ip_Types.h.

6.3.4 Enum Reference

6.3.4.1 Lpuart_Uart_Ip_TransferType

```
enum Lpuart\_Uart\_Ip\_TransferType
```

Type of UART transfer (based on interrupts or DMA).

Enumerator

LPUART_UART_IP_USING_DMA	The driver will use DMA to perform UART transfer.
LPUART_UART_IP_USING_INTERRUPTS	The driver will use interrupts to perform UART transfer.

Definition at line 102 of file Lpuart_Uart_Ip_Types.h.

6.3.4.2 Lpuart_Uart_Ip_StatusType

enum [Lpuart_Uart_Ip_StatusType](#)

Driver status type.

Enumerator

LPUART_UART_IP_STATUS_SUCCESS	Generic operation success status.
LPUART_UART_IP_STATUS_ERROR	Generic operation failure status.
LPUART_UART_IP_STATUS_BUSY	Generic operation busy status.
LPUART_UART_IP_STATUS_TIMEOUT	Generic operation timeout status.
LPUART_UART_IP_STATUS_TX_UNDERRUN	TX underrun error.
LPUART_UART_IP_STATUS_RX_OVERRUN	RX overrun error.
LPUART_UART_IP_STATUS_ABORTED	A transfer was aborted
LPUART_UART_IP_STATUS_FRAMING_ERROR	Framing error.
LPUART_UART_IP_STATUS_PARITY_ERROR	Parity error.
LPUART_UART_IP_STATUS_NOISE_ERROR	Noise error.
LPUART_UART_IP_STATUS_DMA_ERROR	DMA error.

Definition at line 114 of file Lpuart_Uart_Ip_Types.h.

6.3.4.3 Lpuart_Uart_Ip_EventType

enum [Lpuart_Uart_Ip_EventType](#)

Define the enum of the Events which can trigger UART callback.

This enum should include the Events for all platforms

Enumerator

LPUART_UART_IP_EVENT_RX_FULL	Rx buffer is full.
LPUART_UART_IP_EVENT_TX_EMPTY	Tx buffer is empty.
LPUART_UART_IP_EVENT_END_TRANSFER	The current transfer is ending.
LPUART_UART_IP_EVENT_ERROR	An error occurred during transfer.

Definition at line 140 of file Lpuart_Uart_Ip_Types.h.

6.3.4.4 Lpuart_Uart_Ip_BaudrateType

enum [Lpuart_Uart_Ip_BaudrateType](#)

Define the enum of the baudrate values that should be set for Uart.

Definition at line 156 of file Lpuart_Uart_Ip_Types.h.

6.3.5 Function Reference

6.3.5.1 Lpuart_Uart_Ip_Init()

```
void Lpuart_Uart_Ip_Init (
    const uint8 Instance,
    const Lpuart\_Uart\_Ip\_UserConfigType * UserConfig )
```

Initializes an LPUART operation instance.

The caller provides memory for the driver state structures during initialization. The user must select the LPUART clock source in the application to initialize the LPUART.

Parameters

<i>Instance</i>	LPUART instance number
<i>UserConfig</i>	user configuration structure of type Lpuart_Uart_Ip_UserConfigType

Returns

void

6.3.5.2 Lpuart_Uart_Ip_Deinit()

```
Lpuart\_Uart\_Ip\_StatusType Lpuart_Uart_Ip_Deinit (
    const uint8 Instance )
```

Shuts down the LPUART by disabling interrupts and transmitter/receiver.

Parameters

<i>Instance</i>	LPUART instance number
-----------------	------------------------

Returns

LPUART_UART_IP_STATUS_SUCCESS if successful; LPUART_UART_IP_STATUS_ERROR if the progress has not fully completed;

6.3.5.3 Lpuart_Uart_Ip_SyncSend()

```
Lpuart\_Uart\_Ip\_StatusType Lpuart_Uart_Ip_SyncSend (
    const uint8 Instance,
    const uint8 * TxBuff,
    const uint32 TxSize,
    const uint32 Timeout )
```

Send out multiple bytes of data using polling method.

Parameters

<i>Instance</i>	LPUART instance number.
<i>TxBuff</i>	The buffer pointer which saves the data to be sent.
<i>TxSize</i>	Size of data to be sent in unit of byte.
<i>Timeout</i>	value in microseconds.

Returns

LPUART_UART_IP_STATUS_SUCCESS if successful; LPUART_UART_IP_STATUS_BUSY if the resource is busy; LPUART_UART_IP_STATUS_TIMEOUT if timeout occur

6.3.5.4 Lpuart_Uart_Ip_AsyncSend()

```
Lpuart_Uart_Ip_StatusType Lpuart_Uart_Ip_AsyncSend (
    const uint8 Instance,
    const uint8 * TxBuff,
    const uint32 TxSize )
```

Sends data out through the LPUART module using a non-blocking method. This enables an a-sync method for transmitting data. When used with a non-blocking receive, the LPUART can perform a full duplex operation. Non-blocking means that the function returns immediately. The application has to get the transmit status to know when the transmit is complete.

Parameters

<i>Instance</i>	LPUART instance number.
<i>TxBuff</i>	The buffer pointer which saves the data to be sent.
<i>TxSize</i>	Size of data to be sent in unit of byte.

Returns

LPUART_UART_IP_STATUS_SUCCESS if successful; LPUART_UART_IP_STATUS_BUSY if the resource is busy;

6.3.5.5 Lpuart_Uart_Ip_GetTransmitStatus()

```
Lpuart_Uart_Ip_StatusType Lpuart_Uart_Ip_GetTransmitStatus (
    const uint8 Instance,
    uint32 * BytesRemaining )
```

Returns whether the previous transmit is complete.

Parameters

<i>Instance</i>	LPUART instance number
<i>BytesRemaining</i>	Pointer to value that is populated with the number of bytes that have been sent in the active transfer

Note

In DMA mode, this parameter may not be accurate, in case the transfer completes right after calling this function; in this edge-case, the parameter will reflect the initial transfer size, due to automatic reloading of the major loop count in the DMA transfer descriptor.

Returns

The transmit status.

Return values

<i>LPUART_UART_IP_STATUS_SUCCESS</i>	The transmit has completed successfully.
<i>LPUART_UART_IP_STATUS_BUSY</i>	The transmit is still in progress. <i>bytesRemaining</i> will be filled with the number of bytes that are yet to be transmitted.
<i>LPUART_UART_IP_STATUS_ABORTED</i>	The transmit was aborted.
<i>LPUART_UART_IP_STATUS_TIMEOUT</i>	A timeout was reached.
<i>LPUART_UART_IP_STATUS_ERROR</i>	An error occurred.

6.3.5.6 Lpuart_Uart_Ip_AbortSendingData()

```
Lpuart_Uart_Ip_StatusType Lpuart_Uart_Ip_AbortSendingData (
    const uint8 Instance )
```

Terminates a non-blocking transmission early.

Parameters

<i>instance</i>	LPUART instance number
-----------------	------------------------

Returns

LPUART_UART_IP_STATUS_ERROR if the transmit process has not fully completed, LPUART_UART_IP_STATUS_SUCCESS if the transmit process has successfully completed

6.3.5.7 Lpuart_Uart_Ip_SyncReceive()

```
Lpuart_Uart_Ip_StatusType Lpuart_Uart_Ip_SyncReceive (
    const uint8 Instance,
    uint8 * RxBuff,
    const uint32 RxSize,
    const uint32 Timeout )
```

Receive multiple bytes of data using polling method.

Parameters

<i>Instance</i>	LPUART instance number.
<i>RxBuff</i>	The buffer pointer which saves the data to be received.
<i>RxSize</i>	Size of data need to be received in unit of byte.
<i>Timeout</i>	value in microseconds.

Returns

LPUART_UART_IP_STATUS_SUCCESS if the transaction is successful; LPUART_UART_IP_STATUS_BUSY if the resource is busy; LPUART_UART_IP_STATUS_RX_OVERRUN if an overrun error occurred LPUART_UART_IP_STATUS_FRAMING_ERROR if a framing error occurred LPUART_UART_IP_STATUS_PARITY_ERROR if a parity error occurred LPUART_UART_IP_STATUS_NOISE_ERROR if a noise error occurred LPUART_UART_IP_STATUS_TIMEOUT if timeout occur

6.3.5.8 Lpuart_Uart_Ip_AsyncReceive()

```
Lpuart_Uart_Ip_StatusType Lpuart_Uart_Ip_AsyncReceive (
    const uint8 Instance,
    uint8 * RxBuff,
    const uint32 RxSize )
```

Gets data from the LPUART module by using a non-blocking method. This enables an a-sync method for receiving data. When used with a non-blocking transmission, the LPUART can perform a full duplex operation. Non-blocking means that the function returns immediately. The application has to get the receive status to know when the receive is complete.

Parameters

<i>Instance</i>	LPUART instance number
<i>RxBuff</i>	buffer containing 8-bit read data chars received
<i>RxSize</i>	the number of bytes to receive

Returns

LPUART_UART_IP_STATUS_SUCCESS if successful; LPUART_UART_IP_STATUS_BUSY if the resource is busy

6.3.5.9 Lpuart_Uart_Ip_GetReceiveStatus()

```
Lpuart_Uart_Ip_StatusType Lpuart_Uart_Ip_GetReceiveStatus (
    const uint8 Instance,
    uint32 * BytesRemaining )
```

Returns whether the previous receive is complete.

Parameters

<i>Instance</i>	LPUART instance number
<i>BytesRemaining</i>	pointer to value that is filled with the number of bytes that still need to be received in the active transfer.

Note

In DMA mode, this parameter may not be accurate, in case the transfer completes right after calling this function; in this edge-case, the parameter will reflect the initial transfer size, due to automatic reloading of the major loop count in the DMA transfer descriptor.

Returns

The receive status.

Return values

<i>LPUART_UART_IP_STATUS_SUCCESS</i>	the receive has completed successfully.
<i>LPUART_UART_IP_STATUS_BUSY</i>	the receive is still in progress. <i>bytesReceived</i> will be filled with the number of bytes that have been received so far.
<i>LPUART_UART_IP_STATUS_ABORTED</i>	The receive was aborted.
<i>LPUART_UART_IP_STATUS_TIMEOUT</i>	A timeout was reached.
<i>LPUART_UART_IP_STATUS_RX_OVERRUN</i> , <i>LPUART_UART_IP_STATUS_FRAMING_ERROR</i> , <i>LPUART_UART_IP_STATUS_PARITY_ERROR</i> , or <i>Y_ERROR</i>	<i>LPUART_UART_IP_STATUS_NOISE_ERROR</i> , <i>LPUART_UART_IP_STATUS_ERROR</i> An error occurred during reception.

6.3.5.10 Lpuart_Uart_Ip_AbortReceivingData()

```
Lpuart_Uart_Ip_StatusType Lpuart_Uart_Ip_AbortReceivingData (
    const uint8 Instance )
```

Terminates a non-blocking receive early.

Parameters

<i>Instance</i>	LPUART instance number
-----------------	------------------------

Returns

LPUART_UART_IP_STATUS_ERROR if the receive process has not fully completed, *LPUART_UART_IP_STATUS_SUCCESS* if the receive process has successfully completed

6.3.5.11 Lpuart_Uart_Ip_SetBaudRate()

```
Lpuart_Uart_Ip_StatusType Lpuart_Uart_Ip_SetBaudRate (
    const uint8 Instance,
    const Lpuart_Uart_Ip_BaudrateType DesiredBaudrate,
    const uint32 ClockFrequency )
```

Configures the LPUART baud rate.

This function configures the LPUART baud rate. In some LPUART instances the user must disable the transmitter/receiver before calling this function. Generally, this may be applied to all LPUARTs to ensure safe operation.

Parameters

<i>Instance</i>	LPUART instance number.
<i>DesiredBaudrate</i>	LPUART desired baud rate.
<i>ClockFrequency</i>	Clock Frequency of LPUART instance.

Returns

LPUART_UART_IP_STATUS_BUSY if called during an on-going transfer, LPUART_UART_IP_STATUS_SUCCESS otherwise

6.3.5.12 Lpuart_Uart_Ip_GetBaudRate()

```
void Lpuart_Uart_Ip_GetBaudRate (
    const uint8 Instance,
    uint32 * ConfiguredBaudRate )
```

Returns the LPUART baud rate.

This function returns the LPUART configured baud rate.

Parameters

	<i>Instance</i>	LPUART instance number.
out	<i>ConfiguredBaudRate</i>	LPUART configured baud rate.

6.3.5.13 Lpuart_Uart_Ip_SetTxBuffer()

```
void Lpuart_Uart_Ip_SetTxBuffer (
    const uint8 Instance,
    const uint8 * TxBuff,
    const uint32 TxSize )
```

Sets the internal driver reference to the tx buffer.

This function can be called from the tx callback to provide the driver with a new buffer, for continuous transmission.

Parameters

<i>Instance</i>	LPUART instance number
<i>TxBuff</i>	source buffer containing 8-bit data chars to send
<i>TxSize</i>	the number of bytes to send

Returns

void

6.3.5.14 Lpuart_Uart_Ip_SetRxBuffer()

```
void Lpuart_Uart_Ip_SetRxBuffer (
    const uint8 Instance,
    uint8 * RxBuff,
    const uint32 RxSize )
```

Sets the internal driver reference to the rx buffer.

This function can be called from the rx callback to provide the driver with a new buffer, for continuous reception.

Module Documentation

Parameters

<i>instance</i>	LPUART instance number
<i>RxBuff</i>	destination buffer containing 8-bit data chars to receive
<i>RxSize</i>	the number of bytes to receive

Returns

void

How to Reach Us:

Home Page:

nxp.com

Web Support:

nxp.com/support

Limited warranty and liability - Information in this document is believed to be accurate and reliable. However, NXP Semiconductors does not give any representations or warranties, expressed or implied, as to the accuracy or completeness of such information and shall have no liability for the consequences of use of such information. NXP Semiconductors takes no responsibility for the content in this document if provided by an information source outside of NXP Semiconductors.

In no event shall NXP Semiconductors be liable for any indirect, incidental, punitive, special or consequential damages (including - without limitation - lost profits, lost savings, business interruption, costs related to the removal or replacement of any products or rework charges) whether or not such damages are based on tort (including negligence), warranty, breach of contract or any other legal theory.

Notwithstanding any damages that customer might incur for any reason whatsoever, NXP Semiconductors' aggregate and cumulative liability towards customer for the products described herein shall be limited in accordance with the Terms and conditions of commercial sale of NXP Semiconductors.

Right to make changes - NXP Semiconductors reserves the right to make changes to information published in this document, including without limitation specifications and product descriptions, at any time and without notice. This document supersedes and replaces all information supplied prior to the publication hereof.

Suitability for use in automotive applications - This NXP product has been qualified for use in automotive applications. If this product is used by customer in the development of, or for incorporation into, products or services (a) used in safety critical applications or (b) in which failure could lead to death, personal injury, or severe physical or environmental damage (such products and services hereinafter referred to as "Critical Applications"), then customer makes the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, safety, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP. As such, customer assumes all risk related to use of any products in Critical Applications and NXP and its suppliers shall not be liable for any such use by customer. Accordingly, customer will indemnify and hold NXP harmless from any claims, liabilities, damages and associated costs and expenses (including attorneys' fees) that NXP may incur related to customer's incorporation of any product in a Critical Application.

Applications - Applications that are described herein for any of these products are for illustrative purposes only. NXP Semiconductors makes no representation or warranty that such applications will be suitable for the specified use without further testing or modification.

Customers are responsible for the design and operation of their applications and products using NXP Semiconductors products, and NXP Semiconductors accepts no liability for any assistance with applications or customer product design. It is customer's sole responsibility to determine whether the NXP Semiconductors product is suitable and fit for the customer's applications and products planned, as well as for the planned application and use of customer's third party customer(s). Customers should provide appropriate design and operating safeguards to minimize the risks associated with their applications and products.

NXP Semiconductors does not accept any liability related to any default, damage, costs or problem which is based on any weakness or default in the customer's applications or products, or the application or use by customer's third party customer(s). Customer is responsible for doing all necessary testing for the customer's applications and products using NXP Semiconductors products in order to avoid a default of the applications and the products or of the application or use by customer's third party customer(s). NXP does not accept any liability in this respect.

Terms and conditions of commercial sale - NXP Semiconductors products are sold subject to the general terms and conditions of commercial sale, as published at <http://www.nxp.com/profile/terms>, unless otherwise agreed in a valid written individual agreement. In case an individual agreement is concluded only the terms and conditions of the respective agreement shall apply. NXP Semiconductors hereby expressly objects to applying the customer's general terms and conditions with regard to the purchase of NXP Semiconductors products by customer.

Export control - This document as well as the item(s) described herein may be subject to export control regulations. Export might require a prior authorization from competent authorities.

Translations — A non-English (translated) version of a document, including the legal information in that document, is for reference only. The English version shall prevail in case of any discrepancy between the translated and English versions.

Security — Customer understands that all NXP products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Customer is responsible for the design and operation of its applications and products throughout their lifecycles to reduce the effect of these vulnerabilities on customer's applications and products. Customer's responsibility also extends to other open and/or proprietary technologies supported by NXP products for use in customer's applications. NXP accepts no liability for any vulnerability. Customer should regularly check security updates from NXP and follow up appropriately.

Customer shall select products with security features that best meet rules, regulations, and standards of the intended application and make the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP.

NXP has a Product Security Incident Response Team (PSIRT) (reachable at <mailto:PSIRT@nxp.com>) that manages the investigation, reporting, and solution release to security vulnerabilities of NXP products.

NXP B.V. - NXP B.V. is not an operating company and it does not distribute or sell products.